

Lecture 4: Regularization, Generalization, and Model Selection

Tao LIN

SoE, Westlake University

September 21, 2026



- 1 From Fitting to Regularization
 - Over-fitting and Under-fitting
- 2 Model Selection and Generalization Gap
 - Data Model and Learning Algorithm
 - Generalization Gap
 - Model Selection
 - Cross-validation
- 3 Bias-Variance Decomposition

Reading materials

- Chapter 3.2, Bishop, Pattern Recognition and Machine Learning
- Chapter 2, Stanford CS-229,
https://cs229.stanford.edu/notes2022fall/main_notes.pdf
- Bias-Variance decomposition by Scott Fortmann-Roe:
<http://scott.fortmann-roe.com/docs/BiasVariance.html>
- Double-descent phenomenon by Mikhail Belkin et al:
<https://www.pnas.org/content/116/32/15849.short>

Reference

- EPFL, CS-433 Machine Learning, https://github.com/epfml/ML_course

Three questions for this lecture

- 1 How can regularization control a flexible regression model?
- 2 How do we select a model without evaluating it on its training data?
- 3 How do bias and variance explain the effect of model complexity?

One thread connects all three questions: training fit is not the final goal; we care about predictions on unseen data.

Table of Contents

- 1 From Fitting to Regularization
 - Over-fitting and Under-fitting
- 2 Model Selection and Generalization Gap
- 3 Bias-Variance Decomposition

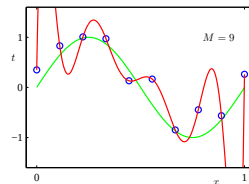
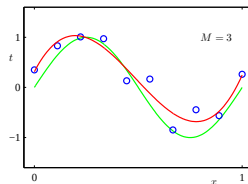
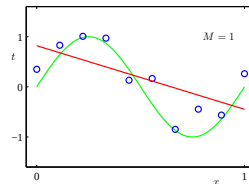
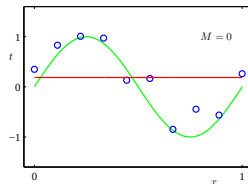
Table of Contents

- 1 From Fitting to Regularization
 - Over-fitting and Under-fitting
- 2 Model Selection and Generalization Gap
 - Data Model and Learning Algorithm
 - Generalization Gap
 - Model Selection
 - Cross-validation
- 3 Bias-Variance Decomposition

Recall: training fit is not unseen-data performance

Let's consider the polynomial regression problem (for a one-dimensional input x_n):

$$\begin{aligned} y_n &\approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M \\ &=: \phi(x_n)^\top \mathbf{w}. \end{aligned} \quad (1)$$

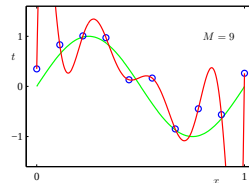
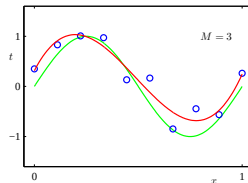
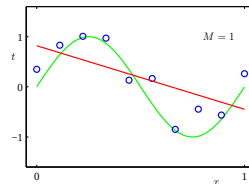
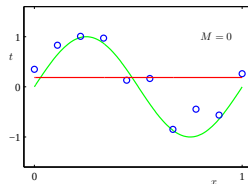


Recall: training fit is not unseen-data performance

Let's consider the polynomial regression problem (for a one-dimensional input x_n):

$$\begin{aligned} y_n &\approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M \\ &=: \phi(x_n)^\top \mathbf{w}. \end{aligned} \quad (1)$$

- **under-fitting**: the family is too inflexible to capture important structure.

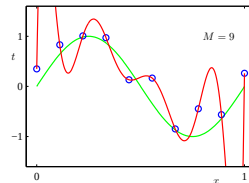
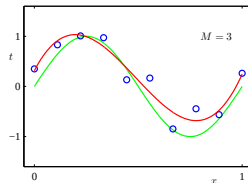
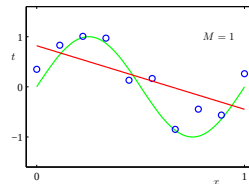
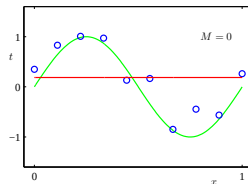


Recall: training fit is not unseen-data performance

Let's consider the polynomial regression problem (for a one-dimensional input x_n):

$$\begin{aligned} y_n &\approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M \\ &=: \phi(x_n)^\top \mathbf{w}. \end{aligned} \quad (1)$$

- **under-fitting**: the family is too inflexible to capture important structure.
- **over-fitting**: the fitted model follows sample-specific variation too closely.

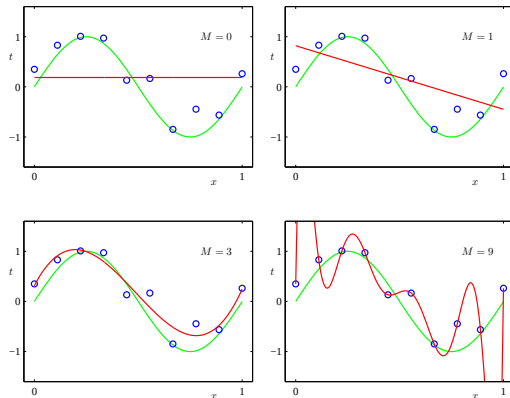


Recall: training fit is not unseen-data performance

Let's consider the polynomial regression problem (for a one-dimensional input x_n):

$$\begin{aligned} y_n &\approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M \\ &=: \phi(x_n)^\top \mathbf{w}. \end{aligned} \quad (1)$$

- **under-fitting**: the family is too inflexible to capture important structure.
- **over-fitting**: the fitted model follows sample-specific variation too closely.



Discussion:

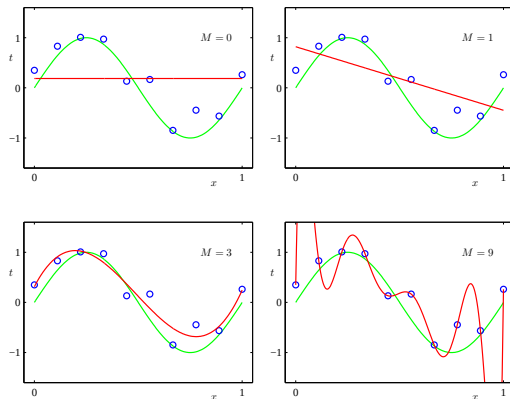
- Both behaviors can lead to poor predictions on unseen data.

Recall: training fit is not unseen-data performance

Let's consider the polynomial regression problem (for a one-dimensional input x_n):

$$\begin{aligned} y_n &\approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M \\ &=: \phi(x_n)^\top \mathbf{w}. \end{aligned} \quad (1)$$

- **under-fitting**: the family is too inflexible to capture important structure.
- **over-fitting**: the fitted model follows sample-specific variation too closely.



Discussion:

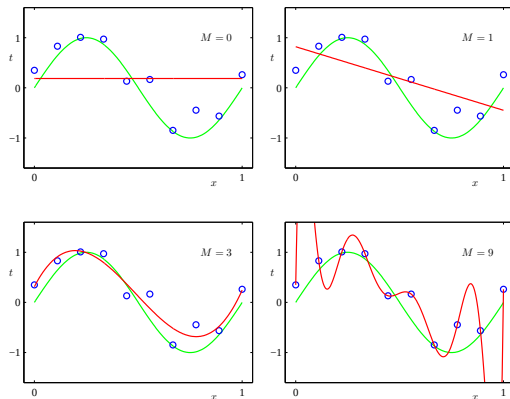
- Both behaviors can lead to poor predictions on unseen data.
- This discussion is made more difficult:

Recall: training fit is not unseen-data performance

Let's consider the polynomial regression problem (for a one-dimensional input x_n):

$$\begin{aligned} y_n &\approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M \\ &=: \phi(x_n)^\top \mathbf{w}. \end{aligned} \quad (1)$$

- **under-fitting**: the family is too inflexible to capture important structure.
- **over-fitting**: the fitted model follows sample-specific variation too closely.



Discussion:

- Both behaviors can lead to poor predictions on unseen data.
- This discussion is made more difficult:
 - since all we have is data.

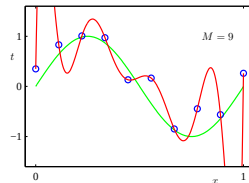
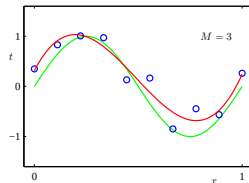
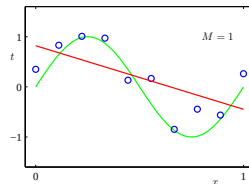
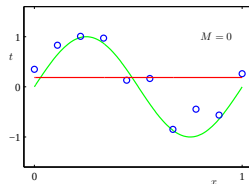
Recall: training fit is not unseen-data performance

Let's consider the polynomial regression problem (for a one-dimensional input x_n):

$$y_n \approx w_0 + w_1 x_n + w_2 x_n^2 + \dots + w_M x_n^M$$

$$=: \phi(x_n)^\top \mathbf{w}.$$
(1)

- **under-fitting**: the family is too inflexible to capture important structure.
- **over-fitting**: the fitted model follows sample-specific variation too closely.



Discussion:

- Both behaviors can lead to poor predictions on unseen data.
- This discussion is made more difficult:
 - since all we have is data.
 - we do not know a priori which variation is stable and which is sample-specific.

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

- Small λ : emphasize training fit; large λ : emphasize shrinkage.

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

- Small λ : emphasize training fit; large λ : emphasize shrinkage.
- **Two common choices:**

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

- Small λ : emphasize training fit; large λ : emphasize shrinkage.
- **Two common choices:**
 - L_2 -regularization (Ridge Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad (3)$$

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

- Small λ : emphasize training fit; large λ : emphasize shrinkage.
- **Two common choices:**
 - L_2 -regularization (Ridge Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad (3)$$

which shrinks coefficients smoothly.

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

- Small λ : emphasize training fit; large λ : emphasize shrinkage.

- **Two common choices:**

- L_2 -regularization (Ridge Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad (3)$$

which shrinks coefficients smoothly.

- L_1 -regularization (Lasso Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1. \quad (4)$$

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

- Small λ : emphasize training fit; large λ : emphasize shrinkage.

- **Two common choices:**

- L_2 -regularization (Ridge Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad (3)$$

which shrinks coefficients smoothly.

- L_1 -regularization (Lasso Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1. \quad (4)$$

which can set some coefficients exactly to zero.

Regularization: trade some training fit for stability

Through [regularization](#), we change which solution is preferred:

$$\min_{\mathbf{w}} \underbrace{\mathcal{L}(\mathbf{w})}_{\text{fit the training data}} + \underbrace{\lambda R(\mathbf{w})}_{\text{penalize large coefficients}}, \quad \lambda \geq 0. \quad (2)$$

- Small λ : emphasize training fit; large λ : emphasize shrinkage.

- **Two common choices:**

- L_2 -regularization (Ridge Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2, \quad (3)$$

which shrinks coefficients smoothly.

- L_1 -regularization (Lasso Regression)

$$\min_{\mathbf{w}} \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_1. \quad (4)$$

which can set some coefficients exactly to zero.

- Standardize features before comparing coefficient penalties; normally the intercept is not penalized.

Ridge regression: the solution changes continuously with λ

For the ridge objective on centered, standardized features,

$$J_{\lambda}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (5)$$

Ridge regression: the solution changes continuously with λ

For the ridge objective on centered, standardized features,

$$J_{\lambda}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (5)$$

Setting the gradient to zero gives

$$(\mathbf{X}^{\top} \mathbf{X} + N\lambda \mathbf{I})\mathbf{w} = \mathbf{X}^{\top} \mathbf{y}. \quad (6)$$

Ridge regression: the solution changes continuously with λ

For the ridge objective on centered, standardized features,

$$J_{\lambda}(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (5)$$

Setting the gradient to zero gives

$$(\mathbf{X}^{\top} \mathbf{X} + N\lambda \mathbf{I})\mathbf{w} = \mathbf{X}^{\top} \mathbf{y}. \quad (6)$$

When the displayed inverse exists,

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}^{\top} \mathbf{X} + N\lambda \mathbf{I})^{-1} \mathbf{X}^{\top} \mathbf{y}. \quad (7)$$

Ridge regression: the solution changes continuously with λ

For the ridge objective on centered, standardized features,

$$J_\lambda(\mathbf{w}) = \frac{1}{2N} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (5)$$

Setting the gradient to zero gives

$$(\mathbf{X}^\top \mathbf{X} + N\lambda \mathbf{I})\mathbf{w} = \mathbf{X}^\top \mathbf{y}. \quad (6)$$

When the displayed inverse exists,

$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}^\top \mathbf{X} + N\lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}. \quad (7)$$

In computation, solve the linear system rather than forming the inverse. Ridge introduces bias, but it can reduce sensitivity to the training sample.

Optional: the probabilistic interpretation of Ridge Regression

A second view; skip on the core classroom route

We start with the posterior $p(\mathbf{w}|\mathbf{X}, \mathbf{y})$ and chose $\mathbf{w}$ to maximize this posterior.

The Maximum-A-Posteriori (MAP) estimate:

$$\mathbf{w}_{\text{ridge}} = \arg \min_{\mathbf{w}} -\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}) \quad (\text{by the definition of posterior})$$

(8)

Optional: the probabilistic interpretation of Ridge Regression

A second view; skip on the core classroom route

We start with the posterior $p(\mathbf{w}|\mathbf{X}, \mathbf{y})$ and chose $\mathbf{w}$ to maximize this posterior.

The Maximum-A-Posteriori (MAP) estimate:

$$\mathbf{w}_{\text{ridge}} = \arg \min_{\mathbf{w}} -\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}) \quad (\text{by the definition of posterior})$$

$$\stackrel{(a)}{=} \arg \min_{\mathbf{w}} -\log \frac{p(\mathbf{y}, \mathbf{X}|\mathbf{w})p(\mathbf{w})}{p(\mathbf{y}, \mathbf{X})} \quad (\text{by the Bayes' law})$$

(8)

Optional: the probabilistic interpretation of Ridge Regression

A second view; skip on the core classroom route

We start with the posterior $p(\mathbf{w}|\mathbf{X}, \mathbf{y})$ and chose $\mathbf{w}$ to maximize this posterior.

The Maximum-A-Posteriori (MAP) estimate:

$$\mathbf{w}_{\text{ridge}} = \arg \min_{\mathbf{w}} -\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}) \quad (\text{by the definition of posterior})$$

$$\stackrel{(a)}{=} \arg \min_{\mathbf{w}} -\log \frac{p(\mathbf{y}, \mathbf{X}|\mathbf{w})p(\mathbf{w})}{p(\mathbf{y}, \mathbf{X})} \quad (\text{by the Bayes' law})$$

$$\stackrel{(b)}{=} \arg \min_{\mathbf{w}} -\log p(\mathbf{y}, \mathbf{X}|\mathbf{w})p(\mathbf{w}) \quad (\text{eliminate quantities that do not depend on } \mathbf{w})$$

(8)

Optional: the probabilistic interpretation of Ridge Regression

A second view; skip on the core classroom route

We start with the posterior $p(\mathbf{w}|\mathbf{X}, \mathbf{y})$ and chose $\mathbf{w}$ to maximize this posterior.

The Maximum-A-Posteriori (MAP) estimate:

$$\begin{aligned}
 \mathbf{w}_{\text{ridge}} &= \arg \min_{\mathbf{w}} -\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}) && \text{(by the definition of posterior)} \\
 &\stackrel{(a)}{=} \arg \min_{\mathbf{w}} -\log \frac{p(\mathbf{y}, \mathbf{X}|\mathbf{w})p(\mathbf{w})}{p(\mathbf{y}, \mathbf{X})} && \text{(by the Bayes' law)} \\
 &\stackrel{(b)}{=} \arg \min_{\mathbf{w}} -\log p(\mathbf{y}, \mathbf{X}|\mathbf{w})p(\mathbf{w}) && \text{(eliminate quantities that do not depend on } \mathbf{w} \text{)} \\
 &\stackrel{(c)}{=} \arg \min_{\mathbf{w}} -\log p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w}) && \text{(eliminate quantities that do not depend on } \mathbf{w} \text{)}
 \end{aligned}
 \tag{8}$$

Optional: the probabilistic interpretation of Ridge Regression

A second view; skip on the core classroom route

We start with the posterior $p(\mathbf{w}|\mathbf{X}, \mathbf{y})$ and chose $\mathbf{w}$ to maximize this posterior.

The Maximum-A-Posteriori (MAP) estimate:

$$\begin{aligned}
 \mathbf{w}_{\text{ridge}} &= \arg \min_{\mathbf{w}} -\log p(\mathbf{w}|\mathbf{X}, \mathbf{y}) && \text{(by the definition of posterior)} \\
 &\stackrel{(a)}{=} \arg \min_{\mathbf{w}} -\log \frac{p(\mathbf{y}, \mathbf{X}|\mathbf{w})p(\mathbf{w})}{p(\mathbf{y}, \mathbf{X})} && \text{(by the Bayes' law)} \\
 &\stackrel{(b)}{=} \arg \min_{\mathbf{w}} -\log p(\mathbf{y}, \mathbf{X}|\mathbf{w})p(\mathbf{w}) && \text{(eliminate quantities that do not depend on } \mathbf{w} \text{)} \\
 &\stackrel{(c)}{=} \arg \min_{\mathbf{w}} -\log p(\mathbf{y}|\mathbf{X}, \mathbf{w})p(\mathbf{w}) && \text{(eliminate quantities that do not depend on } \mathbf{w} \text{)} \\
 &= \arg \min_{\mathbf{w}} \sum_{n=1}^N \frac{1}{2\sigma^2} (y_n - \mathbf{x}_n^\top \mathbf{w})^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2 . && (8)
 \end{aligned}$$

Regularization gives us a family of models.
How should we choose λ ?

Lecture 03 reached:

- Normal Equation and Least Squares
- Probabilistic Interpretation of Linear Regression
- Maximum Likelihood Estimation (MLE)
- Over-fitting and Under-fitting

Lecture 03 reached:

- Normal Equation and Least Squares
- Probabilistic Interpretation of Linear Regression
- Maximum Likelihood Estimation (MLE)
- Over-fitting and Under-fitting

Lecture 04 continues with:

- Regularization, Generalization Gap, and Model Selection
- Bias-Variance Decomposition

Table of Contents

- ① From Fitting to Regularization
- ② **Model Selection and Generalization Gap**
 - Data Model and Learning Algorithm
 - Generalization Gap
 - Model Selection
 - Cross-validation
- ③ Bias-Variance Decomposition

Table of Contents

- ① From Fitting to Regularization
 - Over-fitting and Under-fitting
- ② Model Selection and Generalization Gap
 - Data Model and Learning Algorithm
 - Generalization Gap
 - Model Selection
 - Cross-validation
- ③ Bias-Variance Decomposition

What is the model selection problem?

Recall the Ridge Regression problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \quad + \quad \lambda \|\mathbf{w}\|^2 \quad (9)$$

What is the model selection problem?

Recall the Ridge Regression problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \quad + \quad \lambda \|\mathbf{w}\|^2 \quad (9)$$

- λ can be tuned to control the model complexity



What is the model selection problem?

Recall the Ridge Regression problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \quad + \quad \lambda \|\mathbf{w}\|^2 \quad (9)$$

- λ can be tuned to control the model complexity 
- In practice: $(\lambda_1, \dots, \lambda_k) \longrightarrow \text{Algorithm} \longrightarrow (\mathbf{w}_1, \dots, \mathbf{w}_k)$.

What is the model selection problem?

Recall the Ridge Regression problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \quad + \quad \lambda \|\mathbf{w}\|^2 \quad (9)$$

- λ can be tuned to control the model complexity 
- In practice: $(\lambda_1, \dots, \lambda_k) \longrightarrow \text{Algorithm} \longrightarrow (\mathbf{w}_1, \dots, \mathbf{w}_k)$.
- Which λ should we use?

What is the model selection problem?

Recall the Ridge Regression problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \quad + \quad \lambda \|\mathbf{w}\|^2 \quad (9)$$

- λ can be tuned to control the model complexity 
- In practice: $(\lambda_1, \dots, \lambda_k) \longrightarrow \text{Algorithm} \longrightarrow (\mathbf{w}_1, \dots, \mathbf{w}_k)$.
- Which λ should we use?

Similarly, in Polynomial Regression: $(x_1, x_2) \xrightarrow{\phi} \begin{pmatrix} x_1, x_2, x_1^2 + x_2^2 \\ x_1, x_2, 5x_1^2 + 2x_2^2, x_2^3 + 2x_1 \end{pmatrix}$

What is the model selection problem?

Recall the Ridge Regression problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \quad + \quad \lambda \|\mathbf{w}\|^2 \quad (9)$$

- λ can be tuned to control the model complexity
- In practice: $(\lambda_1, \dots, \lambda_k) \longrightarrow \text{Algorithm} \longrightarrow (\mathbf{w}_1, \dots, \mathbf{w}_k)$.
- Which λ should we use?

Similarly, in Polynomial Regression: $(x_1, x_2) \xrightarrow{\phi} (x_1, x_2, x_1^2 + x_2^2, x_1^2 + 2x_1x_2 + x_2^2, 5x_1^2 + 2x_2^2, x_2^3 + 2x_1x_2)$

- we can enrich the model complexity, by augmenting the feature vector $\mathbf{x}$

What is the model selection problem?

Recall the Ridge Regression problem:

$$\min_{\mathbf{w}} \quad \frac{1}{2N} \sum_{n=1}^N (y_n - \mathbf{x}_n^\top \mathbf{w})^2 \quad + \quad \lambda \|\mathbf{w}\|^2 \quad (9)$$

- λ can be tuned to control the model complexity
- In practice: $(\lambda_1, \dots, \lambda_k) \longrightarrow \text{Algorithm} \longrightarrow (\mathbf{w}_1, \dots, \mathbf{w}_k)$.
- Which λ should we use?

Similarly, in Polynomial Regression: $(x_1, x_2) \xrightarrow{\phi} (x_1, x_2, x_1^2 + x_2^2, x_1^2 + 2x_1x_2 + x_2^2, x_1^3 + 3x_1^2x_2 + 3x_1x_2^2 + x_2^3)$

- we can enrich the model complexity, by augmenting the feature vector $\mathbf{x}$

How do we choose these hyper-parameters?

Model selection for neural networks

Optimization Algorithms?

SGD
Adam
Which step-size?
Which batch-size?
Which momentum?

Neural Architectures?

FullyConnected
ConvNet
ResNet
Transformer
Which width?
Which depth?
Batch normalization?

Regularizations?

Weight decay?
Dropout?
Early stopping?
Data augmentation?

To give a meaningful answer to the above questions,
we first need to specify our data model!

Probabilistic setup

Data model:

- We assume an (unknown) underlying distribution $\mathcal{D}$, with range $\mathcal{X} \times \mathcal{Y}$

Probabilistic setup

Data model:

- We assume an (unknown) underlying distribution $\mathcal{D}$, with range $\mathcal{X} \times \mathcal{Y}$
- We see a dataset S of independent samples from $\mathcal{D}$:

$$S = \{(\mathbf{x}_n, y_n) \mid \text{i.i.d.} \sim \mathcal{D}\}_{n=1}^N. \quad (10)$$

Probabilistic setup

Data model:

- We assume an (unknown) underlying distribution $\mathcal{D}$, with range $\mathcal{X} \times \mathcal{Y}$
- We see a dataset S of independent samples from $\mathcal{D}$:

$$S = \{(\mathbf{x}_n, y_n) \mid \text{i.i.d.} \sim \mathcal{D}\}_{n=1}^N. \quad (10)$$

Learning algorithm:

$$f_S = \mathcal{A}(S) \quad (11)$$

Probabilistic setup

Data model:

- We assume an (unknown) underlying distribution $\mathcal{D}$, with range $\mathcal{X} \times \mathcal{Y}$
- We see a dataset S of independent samples from $\mathcal{D}$:

$$S = \{(\mathbf{x}_n, y_n) \mid \text{i.i.d.} \sim \mathcal{D}\}_{n=1}^N. \quad (10)$$

Learning algorithm:

$$f_S = \mathcal{A}(S) \quad (11)$$

- f_S denotes the output.



Probabilistic setup

Data model:

- We assume an (unknown) underlying distribution $\mathcal{D}$, with range $\mathcal{X} \times \mathcal{Y}$
- We see a dataset S of independent samples from $\mathcal{D}$:

$$S = \{(\mathbf{x}_n, y_n) \mid \text{i.i.d.} \sim \mathcal{D}\}_{n=1}^N. \quad (10)$$

Learning algorithm:

$$f_S = \mathcal{A}(S) \quad (11)$$

- f_S denotes the output.
- $\mathcal{A}$ denotes the learning algorithm.

Probabilistic setup

Data model:

- We assume an (unknown) underlying distribution $\mathcal{D}$, with range $\mathcal{X} \times \mathcal{Y}$
- We see a dataset S of independent samples from $\mathcal{D}$:

$$S = \{(\mathbf{x}_n, y_n) \mid \text{i.i.d.} \sim \mathcal{D}\}_{n=1}^N. \quad (10)$$

Learning algorithm:

$$f_S = \mathcal{A}(S) \quad (11)$$

- f_S denotes the output. 
- $\mathcal{A}$ denotes the learning algorithm. 
- S denotes the input (dataset). 

Probabilistic setup

Data model:

- We assume an (unknown) underlying distribution $\mathcal{D}$, with range $\mathcal{X} \times \mathcal{Y}$
- We see a dataset S of independent samples from $\mathcal{D}$:

$$S = \{(\mathbf{x}_n, y_n) \text{ i.i.d. } \sim \mathcal{D}\}_{n=1}^N. \quad (10)$$

Learning algorithm:

$$f_S = \mathcal{A}(S) \quad (11)$$

- f_S denotes the output. 
- $\mathcal{A}$ denotes the learning algorithm. 
- S denotes the input (dataset). 
- Can add a subscript $f_{S,\lambda}$ to indicate the model dependency (for Ridge Regression).

Population risk: what we ultimately care about

Given a model f , how can we assess if f is any good?

Population risk: what we ultimately care about

Given a model f , how can we assess if f is any good?

- We should compute the **expected error** over all samples chosen according to $\mathcal{D}$:

$$\mathcal{L}_{\mathcal{D}}(f) = \mathbb{E}_{\mathcal{D}} [\ell(y, f(\mathbf{x}))] \quad \text{where } \ell(\cdot, \cdot) \text{ is our loss function} \quad (12)$$

Population risk: what we ultimately care about

Given a model f , how can we assess if f is any good?

- We should compute the **expected error** over all samples chosen according to $\mathcal{D}$:

$$\mathcal{L}_{\mathcal{D}}(f) = \mathbb{E}_{\mathcal{D}} [\ell(y, f(\mathbf{x}))] \quad \text{where } \ell(\cdot, \cdot) \text{ is our loss function} \quad (12)$$

- The quantity $\mathcal{L}_{\mathcal{D}}(f)$ has many names $\left\{ \begin{array}{l} \textit{True} \\ \textit{Expected} \end{array} \right\} \left\{ \begin{array}{l} \textit{Risk} \\ \textit{Error} \\ \textit{Loss} \end{array} \right.$

Population risk: what we ultimately care about

Given a model f , how can we assess if f is any good?

- We should compute the **expected error** over all samples chosen according to $\mathcal{D}$:

$$\mathcal{L}_{\mathcal{D}}(f) = \mathbb{E}_{\mathcal{D}} [\ell(y, f(\mathbf{x}))] \quad \text{where } \ell(\cdot, \cdot) \text{ is our loss function} \quad (12)$$

- The quantity $\mathcal{L}_{\mathcal{D}}(f)$ has many names $\left\{ \begin{array}{l} \textit{True} \\ \textit{Expected} \end{array} \right\} \left\{ \begin{array}{l} \textit{Risk} \\ \textit{Error} \\ \textit{Loss} \end{array} \right.$

This is the quantity we are fundamentally interested in

Population risk: what we ultimately care about

Given a model f , how can we assess if f is any good?

- We should compute the **expected error** over all samples chosen according to $\mathcal{D}$:

$$\mathcal{L}_{\mathcal{D}}(f) = \mathbb{E}_{\mathcal{D}} [\ell(y, f(\mathbf{x}))] \quad \text{where } \ell(\cdot, \cdot) \text{ is our loss function} \quad (12)$$

- The quantity $\mathcal{L}_{\mathcal{D}}(f)$ has many names $\left\{ \begin{array}{l} \textit{True} \\ \textit{Expected} \end{array} \right\} \left\{ \begin{array}{l} \textit{Risk} \\ \textit{Error} \\ \textit{Loss} \end{array} \right.$

This is the quantity we are fundamentally interested in
but we cannot compute it exactly because $\mathcal{D}$ is unknown.

Empirical Error: what we can compute

- We are given some data S

Empirical Error: what we can compute

- We are given some data S
- We can approximate the true error by **averaging the loss function on the dataset S**

$$L_S(f) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f(\mathbf{x}_n)) . \quad (13)$$

Empirical Error: what we can compute

- We are given some data S
- We can approximate the true error by **averaging the loss function on the dataset S**

$$L_S(f) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f(\mathbf{x}_n)) . \quad (13)$$

- This is called *empirical risk/error/loss*.

Empirical Error: what we can compute

- We are given some data S
- We can approximate the true error by **averaging the loss function on the dataset S**

$$L_S(f) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f(\mathbf{x}_n)) . \quad (13)$$

- This is called *empirical risk/error/loss*.
- The samples are random thus $L_S(f)$ is a random variable.

Empirical Error: what we can compute

- We are given some data S
- We can approximate the true error by **averaging the loss function on the dataset S**

$$L_S(f) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f(\mathbf{x}_n)) . \quad (13)$$

- This is called *empirical risk/error/loss*.
- The samples are random thus $L_S(f)$ is a random variable.
- It is an unbiased estimator of the true error.

Empirical Error: what we can compute

- We are given some data S
- We can approximate the true error by **averaging the loss function on the dataset S**

$$L_S(f) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f(\mathbf{x}_n)) . \quad (13)$$

- This is called *empirical risk/error/loss*.
- The samples are random thus $L_S(f)$ is a random variable.
- It is an unbiased estimator of the true error.
- **Generalization gap:** $|L_{\mathcal{D}}(f) - L_S(f)|$.

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

- The empirical error is called the **training error**:

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

- The empirical error is called the **training error**: when the model has been trained on the same data it is applied to, namely,

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

- The empirical error is called the **training error**: when the model has been trained on the same data it is applied to, namely,
 - If we first learn the model from S , i.e., we compute $f_S = \mathcal{A}(S)$,

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

- The empirical error is called the **training error**: when the model has been trained on the same data it is applied to, namely,
 - If we first learn the model from S , i.e., we compute $f_S = \mathcal{A}(S)$,
 - and then we compute the empirical risk of f_S using the same data S .

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

- The empirical error is called the **training error**: when the model has been trained on the same data it is applied to, namely,
 - If we first learn the model from S , i.e., we compute $f_S = \mathcal{A}(S)$,
 - and then we compute the empirical risk of f_S using the same data S .
 - Then in fact we are computing

$$L_S(f_S) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f_S(\mathbf{x}_n)). \quad (14)$$

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

- The empirical error is called the **training error**: when the model has been trained on the same data it is applied to, namely,
 - If we first learn the model from S , i.e., we compute $f_S = \mathcal{A}(S)$,
 - and then we compute the empirical risk of f_S using the same data S .
 - Then in fact we are computing

$$L_S(f_S) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f_S(\mathbf{x}_n)). \quad (14)$$

- This is called *the training (risk/loss/error)*.

Training Error: what we are minimizing

The prediction function f_S is itself a function of the data S .

Assume that we are given the data S .

- The empirical error is called the **training error**: when the model has been trained on the same data it is applied to, namely,
 - If we first learn the model from S , i.e., we compute $f_S = \mathcal{A}(S)$,
 - and then we compute the empirical risk of f_S using the same data S .
 - Then in fact we are computing

$$L_S(f_S) = \frac{1}{|S|} \sum_{(\mathbf{x}_n, y_n) \in S} \ell(y_n, f_S(\mathbf{x}_n)). \quad (14)$$

- This is called *the training (risk/loss/error)*.
- The reason that $L_S(f_S)$ might not be close to $L_{\mathcal{D}}(f_S)$ is of course over-fitting.

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs
if we train and test the model on the same data.

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs
if we train and test the model on the same data.

- **Problem:** Validating model on the same data subset we trained it on!

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs
if we train and test the model on the same data.

- **Problem:** Validating model on the same data subset we trained it on!
- **Fix:** separate data used for fitting, selection, and final evaluation:

$$S = S_{\text{train}} \dot{\cup} S_{\text{val}} \dot{\cup} S_{\text{test}}. \quad (15)$$

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs
if we train and test the model on the same data.

- **Problem:** Validating model on the same data subset we trained it on!
- **Fix:** separate data used for fitting, selection, and final evaluation:

$$S = S_{\text{train}} \dot{\cup} S_{\text{val}} \dot{\cup} S_{\text{test}}. \quad (15)$$

- 1 We **learn** the function $f_{S_{\text{train}}}$ using the train set S_{train}

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs
if we train and test the model on the same data.

- **Problem:** Validating model on the same data subset we trained it on!
- **Fix:** separate data used for fitting, selection, and final evaluation:

$$S = S_{\text{train}} \dot{\cup} S_{\text{val}} \dot{\cup} S_{\text{test}}. \quad (15)$$

- 1 We **learn** the function $f_{S_{\text{train}}}$ using the train set S_{train}
- 2 We **select** settings using the validation error

$$L_{S_{\text{val}}}(f_{S_{\text{train}}}) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f_{S_{\text{train}}}(\mathbf{x}_n)). \quad (16)$$

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs
if we train and test the model on the same data.

- **Problem:** Validating model on the same data subset we trained it on!
- **Fix:** separate data used for fitting, selection, and final evaluation:

$$S = S_{\text{train}} \dot{\cup} S_{\text{val}} \dot{\cup} S_{\text{test}}. \quad (15)$$

- 1 We **learn** the function $f_{S_{\text{train}}}$ using the train set S_{train}
- 2 We **select** settings using the validation error

$$L_{S_{\text{val}}}(f_{S_{\text{train}}}) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f_{S_{\text{train}}}(\mathbf{x}_n)). \quad (16)$$

- 3 We use S_{test} once, after all choices, for the final evaluation.

Training, validation, and test data have different jobs

How can we address the potential over-fitting problem that occurs
if we train and test the model on the same data.

- **Problem:** Validating model on the same data subset we trained it on!
- **Fix:** separate data used for fitting, selection, and final evaluation:

$$S = S_{\text{train}} \dot{\cup} S_{\text{val}} \dot{\cup} S_{\text{test}}. \quad (15)$$

- 1 We **learn** the function $f_{S_{\text{train}}}$ using the train set S_{train}
- 2 We **select** settings using the validation error

$$L_{S_{\text{val}}}(f_{S_{\text{train}}}) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f_{S_{\text{train}}}(\mathbf{x}_n)). \quad (16)$$

- 3 We use S_{test} once, after all choices, for the final evaluation.
- **Trade-off:** each role now has fewer observations. Cross-validation reuses the training portion more efficiently.

For a fixed predictor, how accurate is validation error?

Assume that we have a model f , and that our loss function $\ell(\cdot, \cdot)$ is bounded.

For a fixed predictor, how accurate is validation error?

Assume that we have a model f , and that our loss function $\ell(\cdot, \cdot)$ is bounded.

- We are given a validation set S_{val} chosen i.i.d. from $\mathcal{D}$
(and this set was not used to construct the fixed predictor f).

For a fixed predictor, how accurate is validation error?

Assume that we have a model f , and that our loss function $\ell(\cdot, \cdot)$ is bounded.

- We are given a validation set S_{val} chosen i.i.d. from $\mathcal{D}$ (and this set was not used to construct the fixed predictor f).
- The validation/empirical error is

$$L_{S_{\text{val}}}(f) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f(\mathbf{x}_n)). \quad (17)$$

For a fixed predictor, how accurate is validation error?

Assume that we have a model f , and that our loss function $\ell(\cdot, \cdot)$ is bounded.

- We are given a validation set S_{val} chosen i.i.d. from $\mathcal{D}$ (and this set was not used to construct the fixed predictor f).
- The validation/empirical error is

$$L_{S_{\text{val}}}(f) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f(\mathbf{x}_n)). \quad (17)$$

- The true error is

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(y, \mathbf{x}) \sim \mathcal{D}} [\ell(y, f(\mathbf{x}))].$$

For a fixed predictor, how accurate is validation error?

Assume that we have a model f , and that our loss function $\ell(\cdot, \cdot)$ is bounded.

- We are given a validation set S_{val} chosen i.i.d. from $\mathcal{D}$ (and this set was not used to construct the fixed predictor f).
- The validation/empirical error is

$$L_{S_{\text{val}}}(f) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f(\mathbf{x}_n)). \quad (17)$$

- The true error is

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(y, \mathbf{x}) \sim \mathcal{D}} [\ell(y, f(\mathbf{x}))].$$

- How far are these apart?

For a fixed predictor, how accurate is validation error?

Assume that we have a model f , and that our loss function $\ell(\cdot, \cdot)$ is bounded.

- We are given a validation set S_{val} chosen i.i.d. from $\mathcal{D}$ (and this set was not used to construct the fixed predictor f).
- The validation/empirical error is

$$L_{S_{\text{val}}}(f) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f(\mathbf{x}_n)). \quad (17)$$

- The true error is

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(y, \mathbf{x}) \sim \mathcal{D}} [\ell(y, f(\mathbf{x}))].$$

- How far are these apart?

Definition 1 (Generalization gap)

The *generalization gap* for this evaluation set is

$$|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|.$$

Table of Contents

- ① From Fitting to Regularization
 - Over-fitting and Under-fitting
- ② **Model Selection and Generalization Gap**
 - Data Model and Learning Algorithm
 - **Generalization Gap**
 - Model Selection
 - Cross-validation
- ③ Bias-Variance Decomposition

Generalization gap: sample average versus population average

- **True Error:**

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(y, \mathbf{x}) \sim \mathcal{D}} [\ell(y, f(\mathbf{x}))]. \quad (18)$$

Generalization gap: sample average versus population average

- **True Error:**

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(y, \mathbf{x}) \sim \mathcal{D}} [\ell(y, f(\mathbf{x}))]. \quad (18)$$

- **Validation/Empirical Error:**

$$L_{S_{\text{val}}}(f) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f(\mathbf{x}_n)). \quad (19)$$

Generalization gap: sample average versus population average

- **True Error:**

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(y, \mathbf{x}) \sim \mathcal{D}} [\ell(y, f(\mathbf{x}))]. \quad (18)$$

- **Validation/Empirical Error:**

$$L_{S_{\text{val}}}(f) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f(\mathbf{x}_n)). \quad (19)$$

- **Generalization Gap:**

$$|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|. \quad (20)$$

Generalization gap: sample average versus population average

- **True Error:**

$$L_{\mathcal{D}}(f) = \mathbb{E}_{(y, \mathbf{x}) \sim \mathcal{D}} [\ell(y, f(\mathbf{x}))]. \quad (18)$$

- **Validation/Empirical Error:**

$$L_{S_{\text{val}}}(f) = \frac{1}{|S_{\text{val}}|} \sum_{(\mathbf{x}_n, y_n) \in S_{\text{val}}} \ell(y_n, f(\mathbf{x}_n)). \quad (19)$$

- **Generalization Gap:**

$$|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|. \quad (20)$$

In expectation, they are the same:

$$L_{\mathcal{D}}(f) = \mathbb{E}_{S_{\text{val}} \sim \mathcal{D}} [L_{S_{\text{val}}}(f)], \quad (21)$$

where the expectation is over fresh validation samples and f is fixed.

The variation of $|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|$ matters.

The variation of $|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|$ matters.

Theorem 2

Given a fixed model f , an i.i.d. validation set $S_{\text{val}} \sim \mathcal{D}$ independent of f , and a loss $\ell(\cdot, \cdot) \in [a, b]$:

$$\Pr \left[|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)| \geq \sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2 |S_{\text{val}}|}} \right] \leq \delta \quad (22)$$

The variation of $|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|$ matters.

Theorem 2

Given a fixed model f , an i.i.d. validation set $S_{\text{val}} \sim \mathcal{D}$ independent of f , and a loss $\ell(\cdot, \cdot) \in [a, b]$:

$$\Pr \left[|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)| \geq \sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2|S_{\text{val}}|}} \right] \leq \delta \quad (22)$$

- Exercise:

The variation of $|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|$ matters.

Theorem 2

Given a fixed model f , an i.i.d. validation set $S_{\text{val}} \sim \mathcal{D}$ independent of f , and a loss $\ell(\cdot, \cdot) \in [a, b]$:

$$\Pr \left[|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)| \geq \sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2|S_{\text{val}}|}} \right] \leq \delta \quad (22)$$

- **Exercise:** The deviation decreases as $\mathcal{O}\left(1/\sqrt{|S_{\text{val}}|}\right)$.

The variation of $|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)|$ matters.

Theorem 2

Given a fixed model f , an i.i.d. validation set $S_{\text{val}} \sim \mathcal{D}$ independent of f , and a loss $\ell(\cdot, \cdot) \in [a, b]$:

$$\Pr \left[|L_{\mathcal{D}}(f) - L_{S_{\text{val}}}(f)| \geq \sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2|S_{\text{val}}|}} \right] \leq \delta \quad (22)$$

- **Exercise:** The deviation decreases as $\mathcal{O}\left(1/\sqrt{|S_{\text{val}}|}\right)$.

⇒ The more data points we have, the more confident we are that the empirical loss we measure is close to the true loss.

Why do you care?

Why do you care?

Given a predictor f and a dataset S , we can control the expected risk:

$$\Pr \left[\underbrace{L_{\mathcal{D}}(f)}_{\text{not computable}} \geq \underbrace{L_{S_{\text{val}}}(f)}_{\text{computable}} + \underbrace{\sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2|S_{\text{val}}|}}}_{\text{deviation}} \right] \leq \delta. \quad (23)$$

Why do you care?

Given a predictor f and a dataset S , we can control the expected risk:

$$\Pr \left[\underbrace{L_{\mathcal{D}}(f)}_{\text{not computable}} \geq \underbrace{L_{S_{\text{val}}}(f)}_{\text{computable}} + \underbrace{\sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2|S_{\text{val}}|}}}_{\text{deviation}} \right] \leq \delta. \quad (23)$$

In words, we can compute a probabilistic upper bound on the true risk.

Why do you care?

Given a predictor f and a dataset S , we can control the expected risk:

$$\Pr \left[\underbrace{L_{\mathcal{D}}(f)}_{\text{not computable}} \geq \underbrace{L_{S_{\text{val}}}(f)}_{\text{computable}} + \underbrace{\sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2 |S_{\text{val}}|}}}_{\text{deviation}} \right] \leq \delta. \quad (23)$$

In words, we can compute a probabilistic upper bound on the true risk.

A concrete example: given a dataset S

Why do you care?

Given a predictor f and a dataset S , we can control the expected risk:

$$\Pr \left[\underbrace{L_{\mathcal{D}}(f)}_{\text{not computable}} \geq \underbrace{L_{S_{\text{val}}}(f)}_{\text{computable}} + \underbrace{\sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2 |S_{\text{val}}|}}}_{\text{deviation}} \right] \leq \delta. \quad (23)$$

In words, we can compute a probabilistic upper bound on the true risk.

A concrete example: given a dataset S

- Split: $S = S_{\text{train}} \dot{\cup} S_{\text{val}}$

Why do you care?

Given a predictor f and a dataset S , we can control the expected risk:

$$\Pr \left[\underbrace{L_{\mathcal{D}}(f)}_{\text{not computable}} \geq \underbrace{L_{S_{\text{val}}}(f)}_{\text{computable}} + \underbrace{\sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2|S_{\text{val}}|}}}_{\text{deviation}} \right] \leq \delta. \quad (23)$$

In words, we can compute a probabilistic upper bound on the true risk.

A concrete example: given a dataset S

- Split: $S = S_{\text{train}} \dot{\cup} S_{\text{val}}$
- Train: $\mathcal{A}(S_{\text{train}}) = f_{S_{\text{train}}}$

Why do you care?

Given a predictor f and a dataset S , we can control the expected risk:

$$\Pr \left[\underbrace{L_{\mathcal{D}}(f)}_{\text{not computable}} \geq \underbrace{L_{S_{\text{val}}}(f)}_{\text{computable}} + \underbrace{\sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2 |S_{\text{val}}|}}}_{\text{deviation}} \right] \leq \delta. \quad (23)$$

In words, we can compute a probabilistic upper bound on the true risk.

A concrete example: given a dataset S

- Split: $S = S_{\text{train}} \dot{\cup} S_{\text{val}}$
- Train: $\mathcal{A}(S_{\text{train}}) = f_{S_{\text{train}}}$
- Validate:

$$\Pr \left[L_{\mathcal{D}}(f_{S_{\text{train}}}) \geq L_{S_{\text{val}}}(f_{S_{\text{train}}}) + \sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2 |S_{\text{val}}|}} \right] \leq \delta. \quad (24)$$

Table of Contents

- ① From Fitting to Regularization
 - Over-fitting and Under-fitting
- ② **Model Selection and Generalization Gap**
 - Data Model and Learning Algorithm
 - Generalization Gap
 - **Model Selection**
 - Cross-validation
- ③ Bias-Variance Decomposition

Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

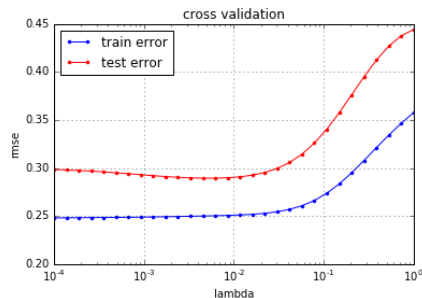
Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

Challenges: We have a set of values $\{\lambda_k\}_{k=1}^K$, which one to choose?

Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

Challenges: We have a set of values $\{\lambda_k\}_{k=1}^K$, which one to choose?

Solutions:

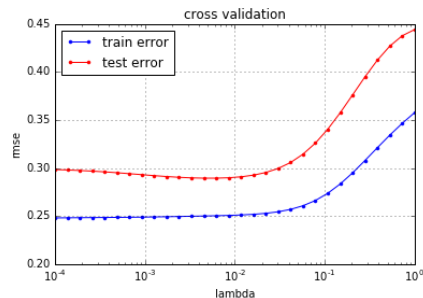


Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

Challenges: We have a set of values $\{\lambda_k\}_{k=1}^K$, which one to choose?

Solutions:

- Split the development data into $S_{\text{train}} \dot{\cup} S_{\text{val}}$.

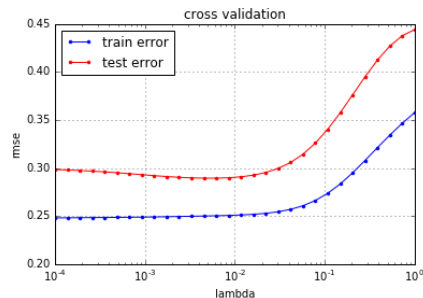


Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

Challenges: We have a set of values $\{\lambda_k\}_{k=1}^K$, which one to choose?

Solutions:

- Split the development data into $S_{\text{train}} \dot{\cup} S_{\text{val}}$.
- Run the learning algorithm K times on the same training set S_{train} to compute the K prediction functions $f_{S_{\text{train}}, \lambda_k}$.

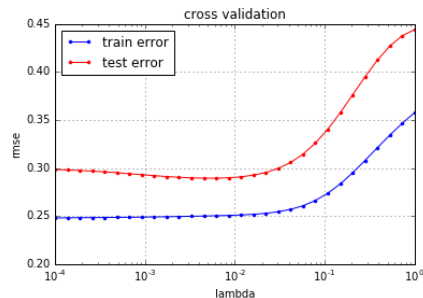


Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

Challenges: We have a set of values $\{\lambda_k\}_{k=1}^K$, which one to choose?

Solutions:

- Split the development data into $S_{\text{train}} \dot{\cup} S_{\text{val}}$.
- Run the learning algorithm K times on the same training set S_{train} to compute the K prediction functions $f_{S_{\text{train}}, \lambda_k}$.
- For each prediction function, compute the validation error $L_{S_{\text{val}}}(f_{S_{\text{train}}, \lambda_k})$.



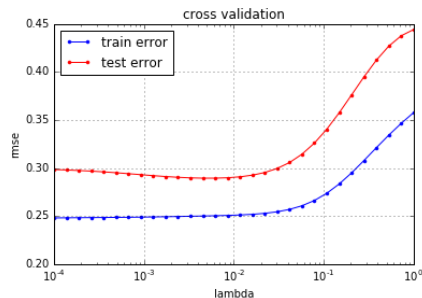
Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

Challenges: We have a set of values $\{\lambda_k\}_{k=1}^K$, which one to choose?

Solutions:

- Split the development data into $S_{\text{train}} \dot{\cup} S_{\text{val}}$.
- Run the learning algorithm K times on the same training set S_{train} to compute the K prediction functions $f_{S_{\text{train}}, \lambda_k}$.
- For each prediction function, compute the validation error $L_{S_{\text{val}}}(f_{S_{\text{train}}, \lambda_k})$.

We choose the value of λ with the smallest validation error.



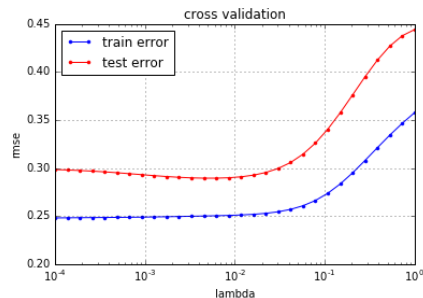
Goal: select the hyper-parameters of our model (e.g., λ for the ridge regression).

Challenges: We have a set of values $\{\lambda_k\}_{k=1}^K$, which one to choose?

Solutions:

- Split the development data into $S_{\text{train}} \dot{\cup} S_{\text{val}}$.
- Run the learning algorithm K times on the same training set S_{train} to compute the K prediction functions $f_{S_{\text{train}}, \lambda_k}$.
- For each prediction function, compute the validation error $L_{S_{\text{val}}}(f_{S_{\text{train}}, \lambda_k})$.

We choose the value of λ with the smallest validation error.



Issue: after comparing many candidates, the smallest validation error can itself be optimistic.

How far are the K validation errors from their true risks?

How far are the K validation errors from their true risks?

Theorem 3

We can bound the maximum deviation for all K candidates, by

$$\Pr \left[\max_k |L_{\mathcal{D}}(f_k) - L_{S_{val}}(f_k)| \geq \sqrt{\frac{(b-a)^2 \ln(2K/\delta)}{2|S_{val}|}} \right] \leq \delta \quad (25)$$

How far are the K validation errors from their true risks?

Theorem 3

We can bound the maximum deviation for all K candidates, by

$$\Pr \left[\max_k |L_{\mathcal{D}}(f_k) - L_{S_{val}}(f_k)| \geq \sqrt{\frac{(b-a)^2 \ln(2K/\delta)}{2|S_{val}|}} \right] \leq \delta \quad (25)$$

- The deviation decreases as $\mathcal{O}(1/\sqrt{|S_{val}|})$ with the number of validation points.

How far are the K validation errors from their true risks?

Theorem 3

We can bound the maximum deviation for all K candidates, by

$$\Pr \left[\max_k |L_{\mathcal{D}}(f_k) - L_{S_{val}}(f_k)| \geq \sqrt{\frac{(b-a)^2 \ln(2K/\delta)}{2|S_{val}|}} \right] \leq \delta \quad (25)$$

- The deviation decreases as $\mathcal{O}(1/\sqrt{|S_{val}|})$ with the number of validation points.
- When testing K hyper-parameters, the error only goes up by $\sqrt{\ln(K)}$.

How far are the K validation errors from their true risks?

Theorem 3

We can bound the maximum deviation for all K candidates, by

$$\Pr \left[\max_k |L_{\mathcal{D}}(f_k) - L_{S_{val}}(f_k)| \geq \sqrt{\frac{(b-a)^2 \ln(2K/\delta)}{2|S_{val}|}} \right] \leq \delta \quad (25)$$

- The deviation decreases as $\mathcal{O}(1/\sqrt{|S_{val}|})$ with the number of validation points.
 - When testing K hyper-parameters, the error only goes up by $\sqrt{\ln(K)}$.
- ⇒ So we can test many different models without incurring a large penalty.

- $k^* = \arg \min_k L_{\mathcal{D}}(f_k)$, i.e., f_{k^*} denotes the function with the smallest true risk.
- $\hat{k} = \arg \min_k L_{S_{\text{val}}}(f_k)$, i.e., $f_{\hat{k}}$ denotes the function with the smallest validation risk.

$$\Pr \left[L_{\mathcal{D}}(f_{\hat{k}}) \geq L_{\mathcal{D}}(f_{k^*}) + 2 \sqrt{\frac{(b-a)^2 \ln(2K/\delta)}{2|S_{\text{val}}|}} \right] \leq \delta \quad (26)$$

- $k^* = \arg \min_k L_{\mathcal{D}}(f_k)$, i.e., f_{k^*} denotes the function with the smallest true risk.
- $\hat{k} = \arg \min_k L_{S_{\text{val}}}(f_k)$, i.e., $f_{\hat{k}}$ denotes the function with the smallest validation risk.

$$\Pr \left[L_{\mathcal{D}}(f_{\hat{k}}) \geq L_{\mathcal{D}}(f_{k^*}) + 2 \sqrt{\frac{(b-a)^2 \ln(2K/\delta)}{2|S_{\text{val}}|}} \right] \leq \delta \quad (26)$$

If we choose the “best” function according to the empirical risk, then its true risk is not too far away from the true risk of the optimal choice.

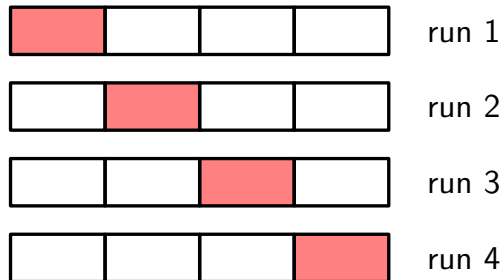
Table of Contents

- ① From Fitting to Regularization
 - Over-fitting and Under-fitting
- ② **Model Selection and Generalization Gap**
 - Data Model and Learning Algorithm
 - Generalization Gap
 - Model Selection
 - **Cross-validation**
- ③ Bias-Variance Decomposition

Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

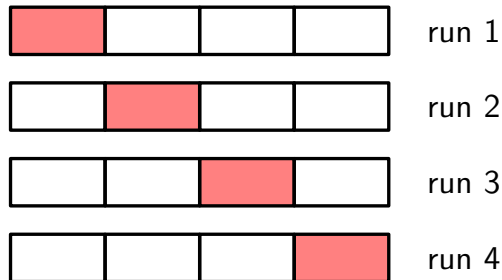
K-fold cross-validation:



Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

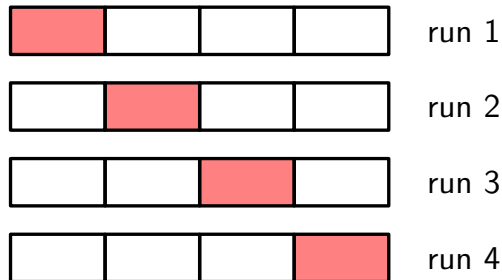
- 1 Randomly partition the data into K groups



Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

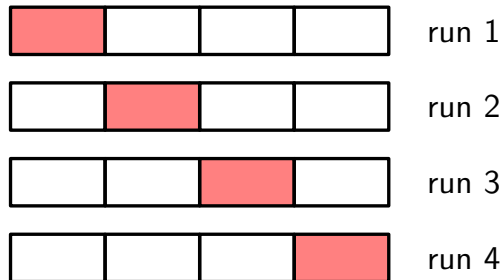
- 1 Randomly partition the data into K groups
- 2 Train K times.



Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

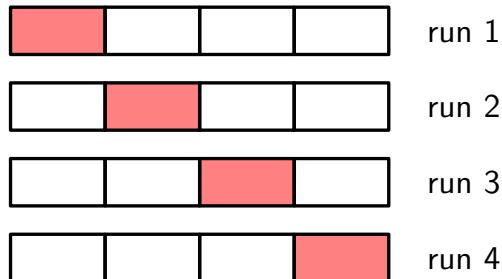
- 1 Randomly partition the data into K groups
- 2 Train K times. Each time leave out exactly one group for validation



Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

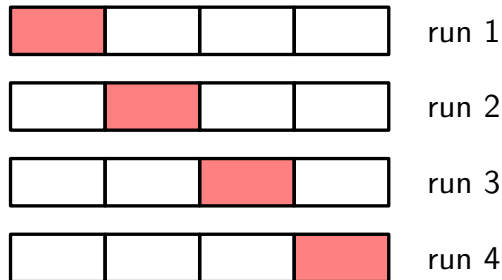
- 1 Randomly partition the data into K groups
- 2 Train K times. Each time leave out exactly one group for validation and use the remaining $K - 1$ groups for training.



Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

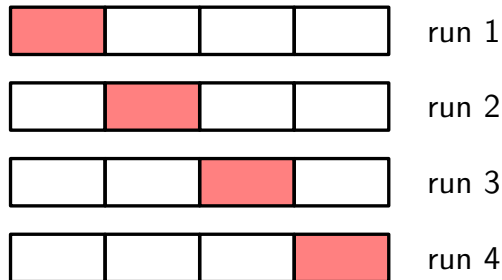
- 1 Randomly partition the data into K groups
- 2 Train K times. Each time leave out exactly one group for validation and use the remaining $K - 1$ groups for training.
- 3 Average the K results



Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

- 1 Randomly partition the data into K groups
- 2 Train K times. Each time leave out exactly one group for validation and use the remaining $K - 1$ groups for training.
- 3 Average the K results

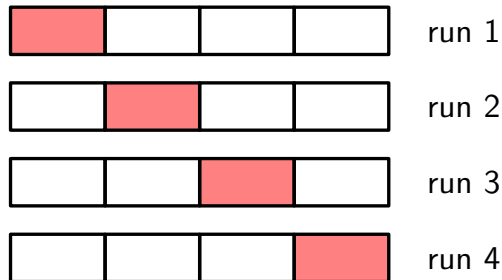


Benefits:

Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

- 1 Randomly partition the data into K groups
- 2 Train K times. Each time leave out exactly one group for validation and use the remaining $K - 1$ groups for training.
- 3 Average the K results



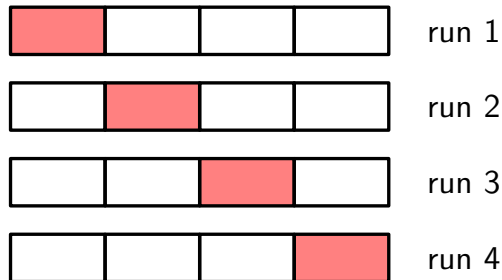
Benefits:

- Every development example is used for training in $K - 1$ folds and for validation once.

Issue: a single train/validation split can be noisy and uses less data for fitting
is not the most efficient way to use the data!

K-fold cross-validation:

- 1 Randomly partition the data into K groups
- 2 Train K times. Each time leave out exactly one group for validation and use the remaining $K - 1$ groups for training.
- 3 Average the K results



Benefits:

- Every development example is used for training in $K - 1$ folds and for validation once.
- The average estimates the performance of this training procedure; it is not exactly unbiased in general. Keep a final test set untouched.

Table of Contents

- 1 From Fitting to Regularization
- 2 Model Selection and Generalization Gap
- 3 Bias-Variance Decomposition**

Previously:

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**
 - How can we judge if a given predictor is good?

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?
- **Solutions:**

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?
- **Solutions:**
 - ⇒ Bound the difference between the true and empirical risks.

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?
- **Solutions:**
 - ⇒ Bound the difference between the true and empirical risks.
 - ⇒ Fit on training data, select on validation data, and evaluate once on test data.

Previously:

- **Motivation:** Hyperparameters search (which often **control the complexity**)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?
- **Solutions:**
 - ⇒ Bound the difference between the true and empirical risks.
 - ⇒ Fit on training data, select on validation data, and evaluate once on test data.

This part:

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?
- **Solutions:**
 - ⇒ Bound the difference between the true and empirical risks.
 - ⇒ Fit on training data, select on validation data, and evaluate once on test data.

This part:

- the role of the complexity of the class

Previously:

- **Motivation:** Hyperparameters search (which often control the complexity)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?
- **Solutions:**
 - ⇒ Bound the difference between the true and empirical risks.
 - ⇒ Fit on training data, select on validation data, and evaluate once on test data.

This part:

- the role of the complexity of the class
- How does the risk behave as a function of the complexity of the model class?

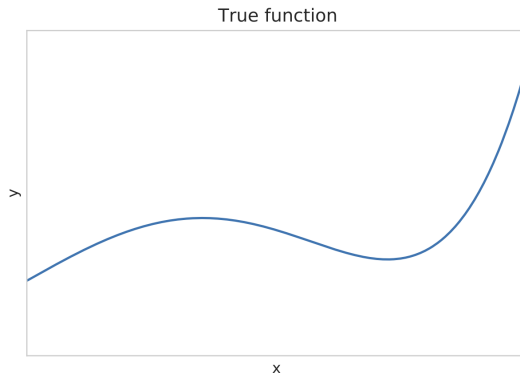
Previously:

- **Motivation:** Hyperparameters search (which often **control the complexity**)
- **Questions:**
 - How can we judge if a given predictor is good?
 - How to select the best models of a family?
- **Solutions:**
 - ⇒ Bound the difference between the true and empirical risks.
 - ⇒ Fit on training data, select on validation data, and evaluate once on test data.

This part:

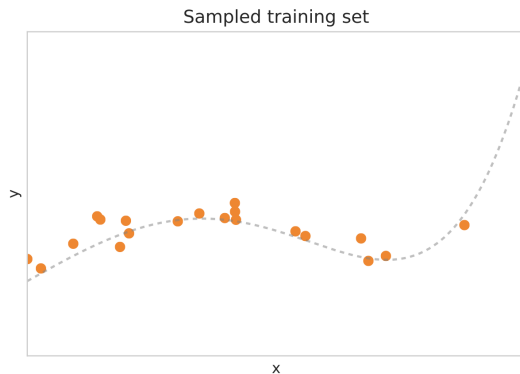
- the **role of the complexity of the class**
- How does the risk behave as a function of the complexity of the model class?
- It will help us to decide how complex and rich we should make our model

Motivation example: 1D-regression



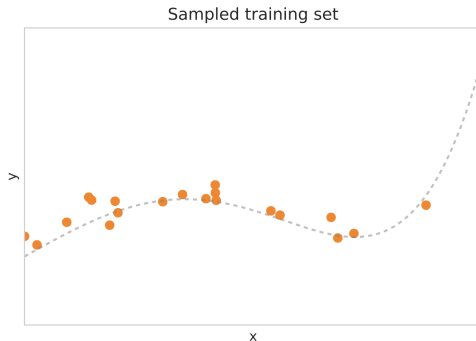
We have a true underlying function in blue we would like to recover.

Motivation example: 1D-regression



We have a true underlying function in blue we would like to recover.

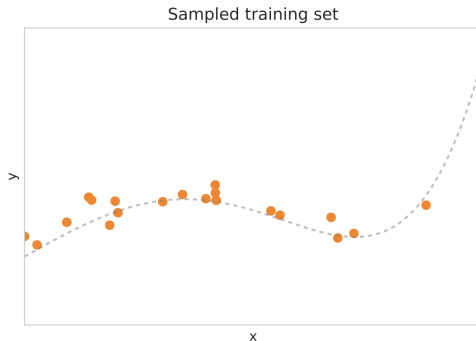
Motivation example: 1D-regression



We have a true underlying function in blue we would like to recover.

- Considering Linear Regression with polynomial feature expansion $x, x^2, x^3, \dots, x^d$.

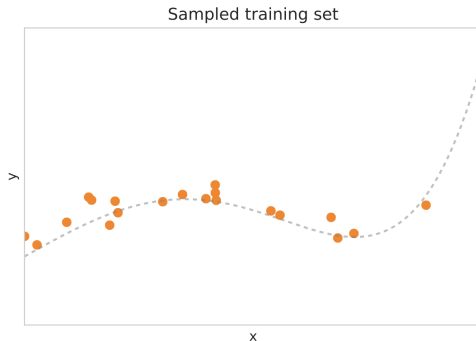
Motivation example: 1D-regression



We have a true underlying function in blue we would like to recover.

- Considering Linear Regression with polynomial feature expansion $x, x^2, x^3, \dots, x^d$.
- The maximum degree d measures the complexity of the class

Motivation example: 1D-regression



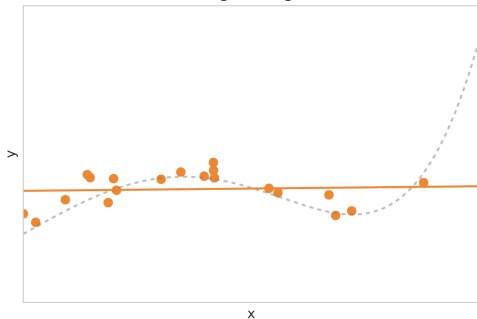
We have a true underlying function in blue we would like to recover.

- Considering Linear Regression with polynomial feature expansion $x, x^2, x^3, \dots, x^d$.
- The maximum degree d measures the complexity of the class
 - ⇒ How far should we go?

Motivation example: 1D-regression

If we restrict ourselves to simple models:

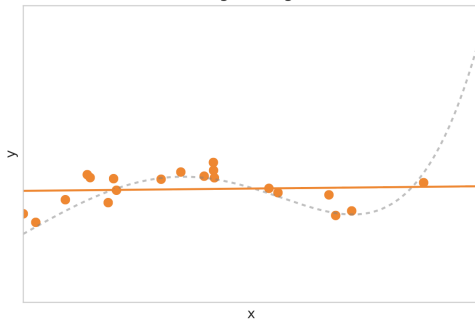
Training fit (degree 1)



Motivation example: 1D-regression

If we restrict ourselves to simple models:

Training fit (degree 1)

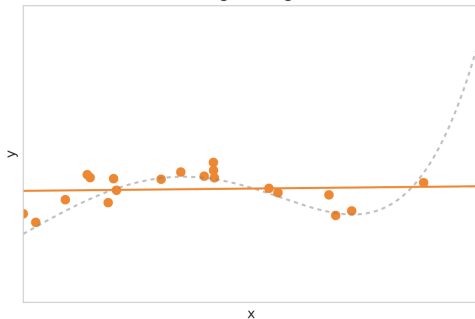


- No linear function would be a good predictor.

Motivation example: 1D-regression

If we restrict ourselves to simple models:

Training fit (degree 1)

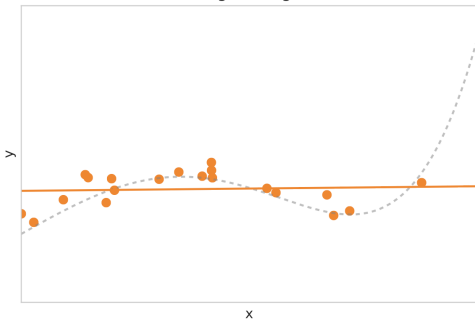


- No linear function would be a good predictor.
- The model is not rich/expressive enough.

Motivation example: 1D-regression

If we restrict ourselves to simple models:

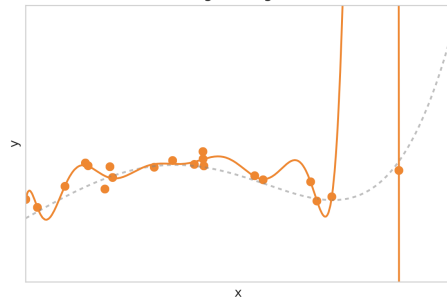
Training fit (degree 1)



- No linear function would be a good predictor.
- The model is not rich/expressive enough.

If we consider a high-degree polynomial:

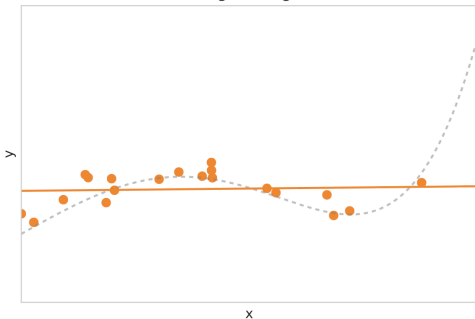
Training fit (degree 12)



Motivation example: 1D-regression

If we restrict ourselves to simple models:

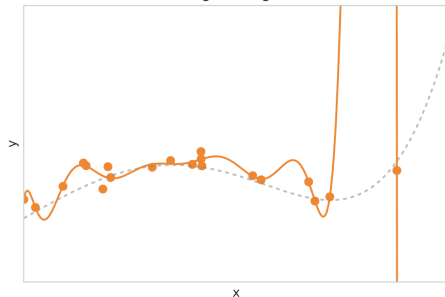
Training fit (degree 1)



- No linear function would be a good predictor.
- The model is not rich/expressive enough.

If we consider a high-degree polynomial:

Training fit (degree 12)

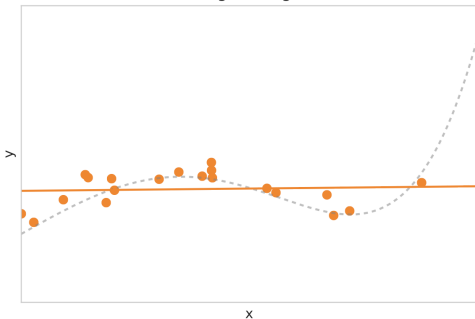


- We can find a model that fits the data well

Motivation example: 1D-regression

If we restrict ourselves to simple models:

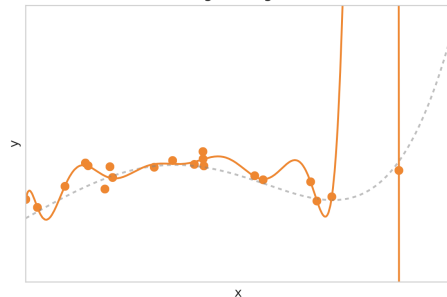
Training fit (degree 1)



- No linear function would be a good predictor.
- The model is not rich/expressive enough.

If we consider a high-degree polynomial:

Training fit (degree 12)

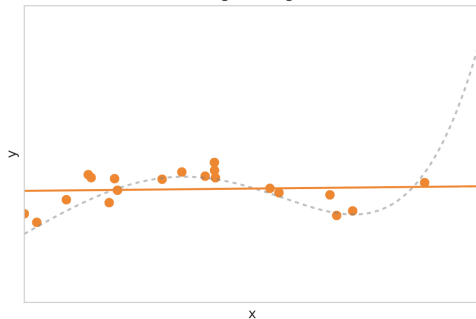


- We can find a model that fits the data well
- Is it a good predictor?

Motivation example: 1D-regression

If we restrict ourselves to simple models:

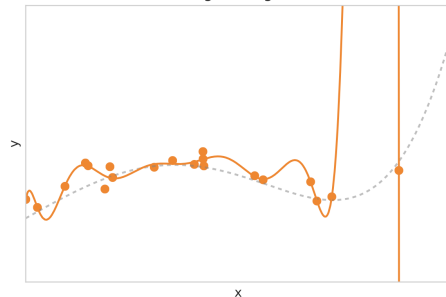
Training fit (degree 1)



- No linear function would be a good predictor.
- The model is not rich/expressive enough.

If we consider a high-degree polynomial:

Training fit (degree 12)

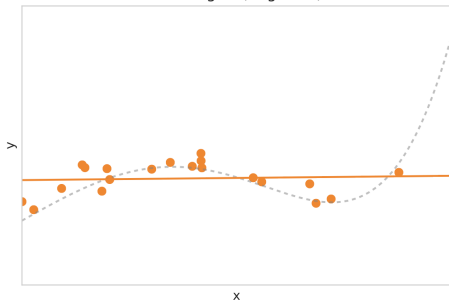


- We can find a model that fits the data well
- Is it a good predictor?
- No: Large generalization error

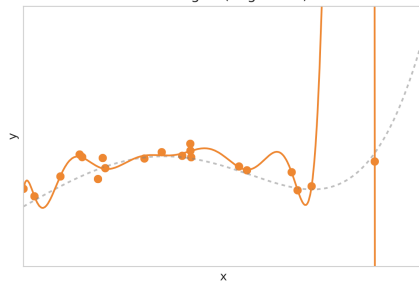
Motivation example: 1D-regression

Simple models are less sensitive.

Training fit (degree 1)



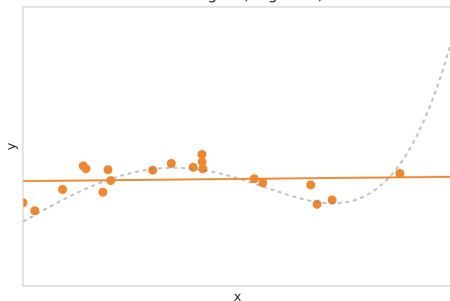
Training fit (degree 12)



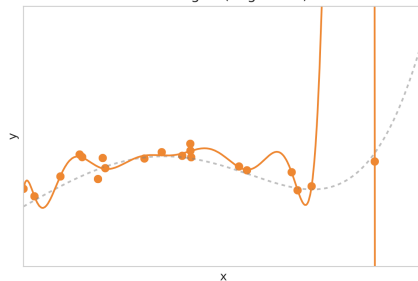
Motivation example: 1D-regression

Simple models are less sensitive.

Training fit (degree 1)



Training fit (degree 12)

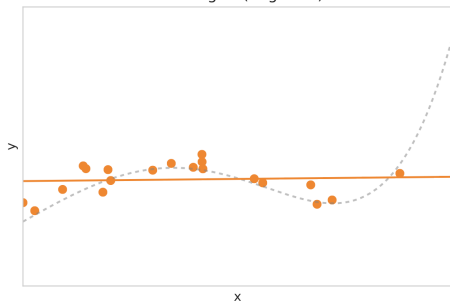


- moving a single observation will cause only a small shift in the position of the line

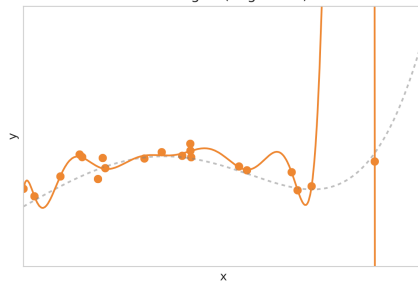
Motivation example: 1D-regression

Simple models are less sensitive.

Training fit (degree 1)



Training fit (degree 12)

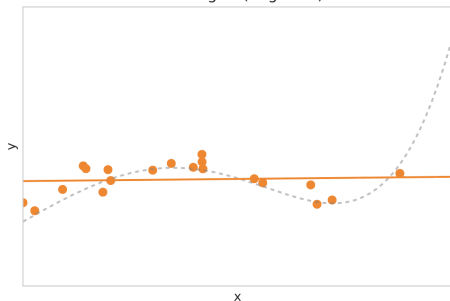


- moving a single observation will cause only a small shift in the position of the line
- under-fitting

Motivation example: 1D-regression

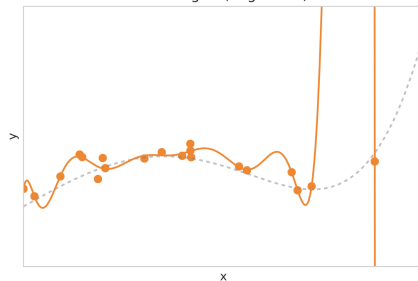
Simple models are less sensitive.

Training fit (degree 1)



- moving a single observation will cause only a small shift in the position of the line
- under-fitting

Training fit (degree 12)

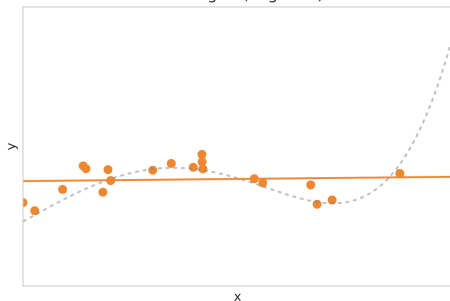


- Changing one of the data points may change the prediction considerable

Motivation example: 1D-regression

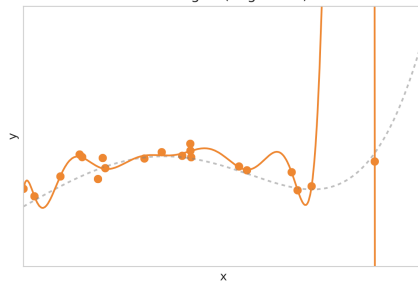
Simple models are less sensitive.

Training fit (degree 1)



- moving a single observation will cause only a small shift in the position of the line
- under-fitting

Training fit (degree 12)

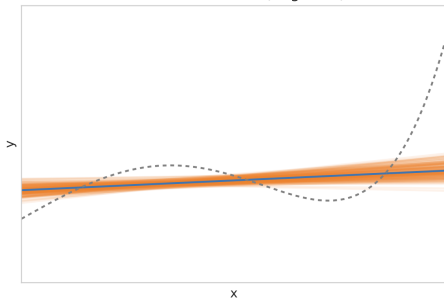


- Changing one of the data points may change the prediction considerable
- over-fitting

Motivation example: 1D-regression

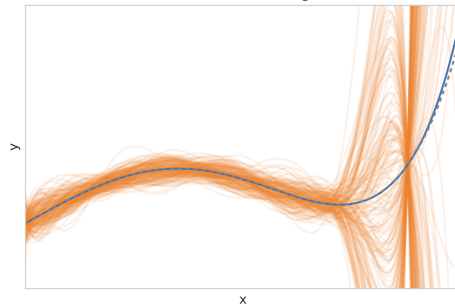
Simple models have large biases but low variance.

Learned functions (degree 1)



Complex models have low bias but high variance.

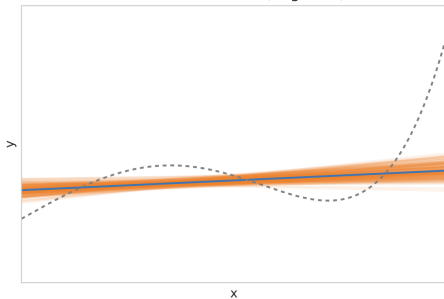
Learned functions (degree 9)



Motivation example: 1D-regression

Simple models have large biases but low variance.

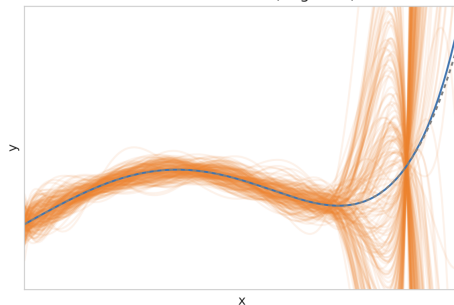
Learned functions (degree 1)



- **large bias**: the average of the predictions f_S does not fit well the data

Complex models have low bias but high variance.

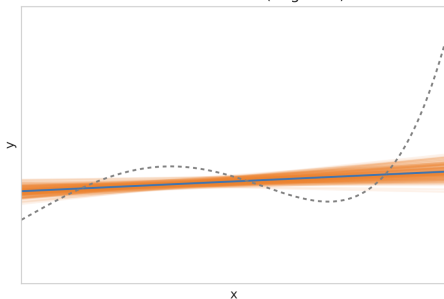
Learned functions (degree 9)



Motivation example: 1D-regression

Simple models have large biases but low variance.

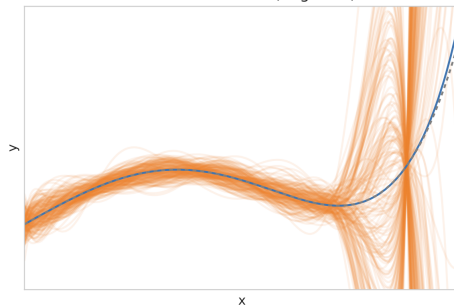
Learned functions (degree 1)



- **large bias**: the average of the predictions f_S does not fit well the data
- **small variance**: the variance of the predictions f_S as a function of S is small

Complex models have low bias but high variance.

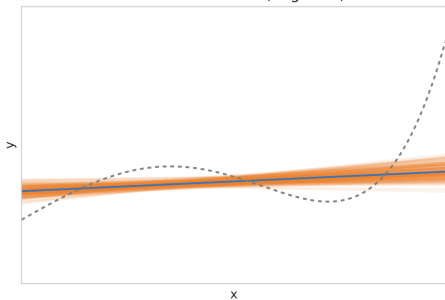
Learned functions (degree 9)



Motivation example: 1D-regression

Simple models have large biases but low variance.

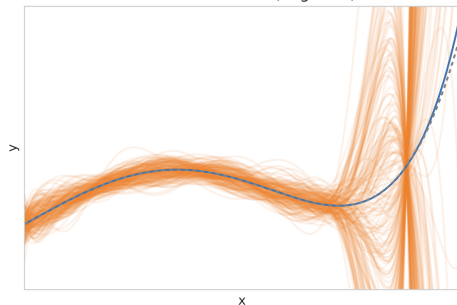
Learned functions (degree 1)



- **large bias**: the average of the predictions f_S does not fit well the data
- **small variance**: the variance of the predictions f_S as a function of S is small

Complex models have low bias but high variance.

Learned functions (degree 9)

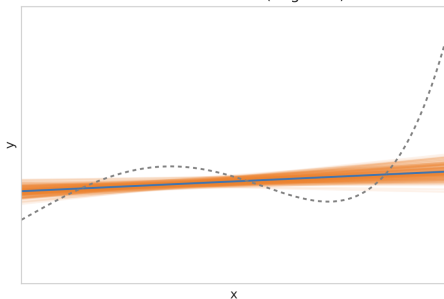


- **small bias**

Motivation example: 1D-regression

Simple models have large biases but low variance.

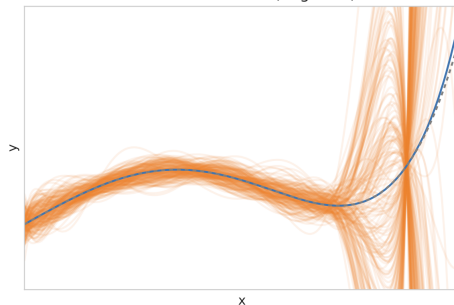
Learned functions (degree 1)



- **large bias**: the average of the predictions f_S does not fit well the data
- **small variance**: the variance of the predictions f_S as a function of S is small

Complex models have low bias but high variance.

Learned functions (degree 9)



- **small bias**
- **large variance**

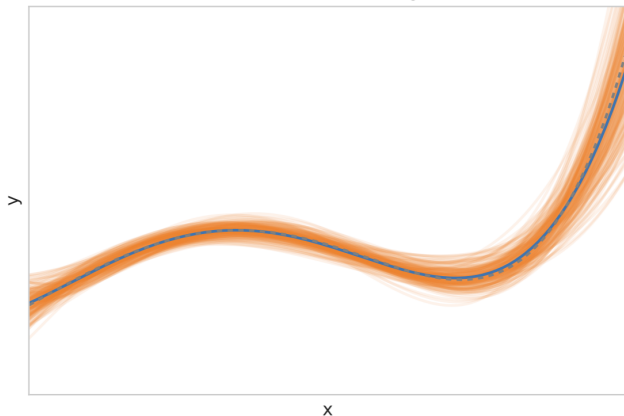
Motivation example: 1D-regression

We need to balance bias & variance correctly

Motivation example: 1D-regression

We need to balance bias & variance correctly

Learned functions (degree 4)



Data model

We assume that the data forms a joint distribution $(\mathbf{x}, y) \sim \mathcal{D}$, and is generated as

$$y = f(x) + \epsilon \quad (27)$$

Data model

We assume that the data forms a joint distribution $(\mathbf{x}, y) \sim \mathcal{D}$, and is generated as

- **True model:** f is an arbitrary and unknown function.

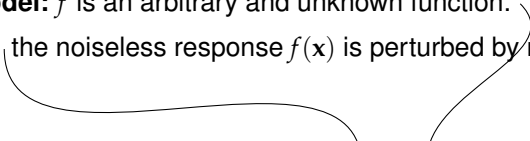

$$y = f(x) + \epsilon$$

(27)

Data model

We assume that the data forms a joint distribution $(\mathbf{x}, y) \sim \mathcal{D}$, and is generated as

- **True model:** f is an arbitrary and unknown function.
- **Output:** the noiseless response $f(\mathbf{x})$ is perturbed by noise.


$$y = f(x) + \epsilon$$

(27)

Data model

We assume that the data forms a joint distribution $(\mathbf{x}, y) \sim \mathcal{D}$, and is generated as

- **True model:** f is an arbitrary and unknown function.
- **Output:** the noiseless response $f(\mathbf{x})$ is perturbed by noise.

$$y = f(x) + \epsilon$$

(27)

- **Input:** $\mathbf{x} \sim \mathcal{D}_X$, the unknown input marginal

Data model

We assume that the data forms a joint distribution $(\mathbf{x}, y) \sim \mathcal{D}$, and is generated as

- **True model:** f is an arbitrary and unknown function.
- **Output:** the noiseless response $f(\mathbf{x})$ is perturbed by noise.

$$y = f(x) + \epsilon$$

The diagram shows the equation $y = f(x) + \epsilon$ with four colored boxes: y (blue), f (red), x (green), and ϵ (teal). Arrows connect the text above to these components: an arrow from 'Output' points to y ; an arrow from 'True model' points to f ; an arrow from 'Input' points to x ; and an arrow from 'Noise' points to ϵ .

(27)

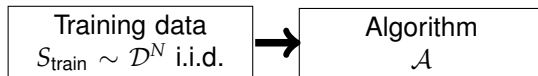
- **Input:** $\mathbf{x} \sim \mathcal{D}_X$, the unknown input marginal
- **Noise:** $\epsilon \sim \mathcal{D}_\epsilon$, independent of $\mathbf{x}$, with $\mathbb{E}[\epsilon] = 0$

Error decomposition

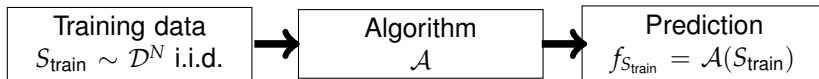
Error decomposition

Training data
 $S_{\text{train}} \sim \mathcal{D}^N$ i.i.d.

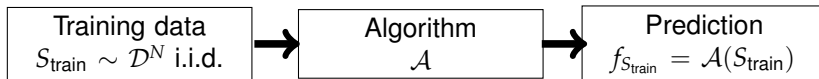
Error decomposition



Error decomposition



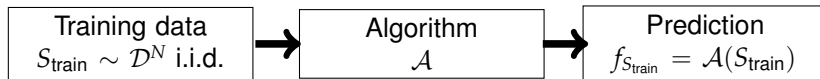
Error decomposition



- We are interested in how the expected error of f_S :

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[(y - f_S(\mathbf{x}))^2 \right] \quad (28)$$

Error decomposition

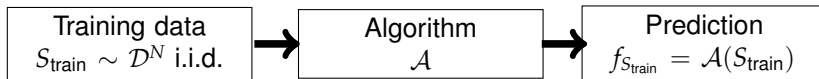


- We are interested in how the expected error of f_S :

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[(y - f_S(\mathbf{x}))^2 \right] \quad (28)$$

behaves as a function of

Error decomposition



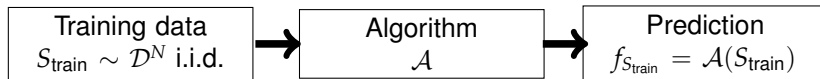
- We are interested in how the expected error of f_S :

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[(y - f_S(\mathbf{x}))^2 \right] \quad (28)$$

behaves as a function of

- 1 the train set S

Error decomposition



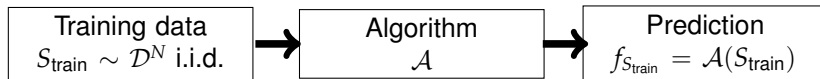
- We are interested in how the expected error of f_S :

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[(y - f_S(\mathbf{x}))^2 \right] \quad (28)$$

behaves as a function of

- 1 the train set S
- 2 the complexity of the model class

Error decomposition



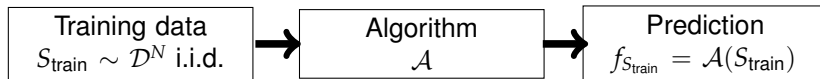
- We are interested in how the expected error of f_S :

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[(y - f_S(\mathbf{x}))^2 \right] \quad (28)$$

behaves as a function of

- 1 the train set S
 - 2 the complexity of the model class
- The decomposition will be true for every single point $\mathbf{x}$.

Error decomposition



- We are interested in how the expected error of f_S :

$$\mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[(y - f_S(\mathbf{x}))^2 \right] \quad (28)$$

behaves as a function of

- 1 the train set S
 - 2 the complexity of the model class
- The decomposition will be true for every single point $\mathbf{x}$.
 - To simplify, we consider the expected error of f_S for a fixed element $\mathbf{x}_0$:

$$L(f_{S_{\text{train}}}) = \mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} \left[(f(\mathbf{x}_0) + \epsilon - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right] \quad (29)$$

A decomposition in three terms

We are interested in the expectation of the true risk over the training set S

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] \tag{30}$$

(31)

(32)

A decomposition in three terms

We are interested in the expectation of the true risk over the training set S

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] \tag{30}$$

$$:= \mathbb{E}_{S_{\text{train}} \in \mathcal{D}, \epsilon \in \mathcal{D}_\epsilon} \left[\left(f(\mathbf{x}_0) + \epsilon - f_{S_{\text{train}}}(\mathbf{x}_0) \right)^2 \right] \tag{31}$$

$$\tag{32}$$

A decomposition in three terms

We are interested in the expectation of the true risk over the training set S

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] \quad (30)$$

$$:= \mathbb{E}_{S_{\text{train}} \in \mathcal{D}, \epsilon \in \mathcal{D}_\epsilon} \left[\left(f(\mathbf{x}_0) + \epsilon - f_{S_{\text{train}}}(\mathbf{x}_0) \right)^2 \right] \quad (31)$$

$$= \mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}, \epsilon \sim \mathcal{D}_\epsilon} [2\epsilon(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (32)$$

A decomposition in three terms

We are interested in the expectation of the true risk over the training set S

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] \quad (30)$$

$$:= \mathbb{E}_{S_{\text{train}} \in \mathcal{D}, \epsilon \in \mathcal{D}_\epsilon} \left[\left(f(\mathbf{x}_0) + \epsilon - f_{S_{\text{train}}}(\mathbf{x}_0) \right)^2 \right] \quad (31)$$

$$= \mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}, \epsilon \sim \mathcal{D}_\epsilon} [2\epsilon(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (32)$$

Using that $\mathbb{E}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon] = 0$ and ϵ is independent from S_{train} :

A decomposition in three terms

We are interested in the expectation of the true risk over the training set S

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] \quad (30)$$

$$:= \mathbb{E}_{S_{\text{train}} \in \mathcal{D}, \epsilon \in \mathcal{D}_\epsilon} \left[\left(f(\mathbf{x}_0) + \epsilon - f_{S_{\text{train}}}(\mathbf{x}_0) \right)^2 \right] \quad (31)$$

$$= \mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}, \epsilon \sim \mathcal{D}_\epsilon} [2\epsilon(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (32)$$

Using that $\mathbb{E}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon] = 0$ and ϵ is independent from S_{train} :

- $\mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] = \text{Var}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon]$

A decomposition in three terms

We are interested in the expectation of the true risk over the training set S

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] \quad (30)$$

$$:= \mathbb{E}_{S_{\text{train}} \in \mathcal{D}, \epsilon \in \mathcal{D}_\epsilon} \left[\left(f(\mathbf{x}_0) + \epsilon - f_{S_{\text{train}}}(\mathbf{x}_0) \right)^2 \right] \quad (31)$$

$$= \mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}, \epsilon \sim \mathcal{D}_\epsilon} [2\epsilon(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (32)$$

Using that $\mathbb{E}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon] = 0$ and ϵ is independent from S_{train} :

- $\mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] = \text{Var}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon]$
- $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}, \epsilon \sim \mathcal{D}_\epsilon} [2\epsilon(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))] = 0.$

A decomposition in three terms

We are interested in the expectation of the true risk over the training set S

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] \quad (30)$$

$$:= \mathbb{E}_{S_{\text{train}} \in \mathcal{D}, \epsilon \in \mathcal{D}_\epsilon} \left[\left(f(\mathbf{x}_0) + \epsilon - f_{S_{\text{train}}}(\mathbf{x}_0) \right)^2 \right] \quad (31)$$

$$= \mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}, \epsilon \sim \mathcal{D}_\epsilon} [2\epsilon(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))] + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (32)$$

Using that $\mathbb{E}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon] = 0$ and ϵ is independent from S_{train} :

- $\mathbb{E}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon^2] = \text{Var}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon]$
- $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}, \epsilon \sim \mathcal{D}_\epsilon} [2\epsilon(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))] = 0$.

Therefore

$$\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] = \underbrace{\text{Var}_{\epsilon \in \mathcal{D}_\epsilon} [\epsilon]}_{\text{noise variance}} + \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (33)$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \tag{34}$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(\text{Bias})^2 + (\text{Variance})^2] \tag{35}$$

$$\tag{37}$$

$$\tag{39}$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \tag{34}$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] +)^2] \tag{35}$$

$$\tag{37}$$

$$\tag{39}$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (34)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] + \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (35)$$

$$(37)$$

$$(39)$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (34)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] + \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (35)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[\right] \quad (36)$$

$$(37)$$

$$(39)$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (34)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] + \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (35)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2 \right] \quad (36)$$

$$(37)$$

$$(39)$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (34)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] + \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (35)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0))]^2 + (\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (36)$$

$$(37)$$

$$(39)$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (34)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] + \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (35)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2 + (\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right] \quad (36)$$

$$+ \underbrace{\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [2 ((f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)]) (\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))]}_0 \quad (37)$$

(39)

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (34)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] + \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (35)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2 + (\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right] \quad (36)$$

$$+ \underbrace{\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [2 ((f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)]) (\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))]}_0 \quad (37)$$

$$= \underbrace{\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2]}_{\text{squared bias}} \quad (38)$$

$$(39)$$

We can further decompose $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2]$ into two terms:

$$\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (34)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] + \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2] \quad (35)$$

$$= \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2 + (\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right] \quad (36)$$

$$+ \underbrace{\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [2 ((f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)]) (\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))]}_0 \quad (37)$$

$$= \underbrace{\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2]}_{\text{squared bias}} \quad (38)$$

$$+ \underbrace{\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} [(\mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2]}_{\text{variance}} \quad (39)$$

Bias-Variance Decomposition

$$\begin{aligned}\mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] &= \text{Var}_{\epsilon \sim \mathcal{D}_{\epsilon}} [\epsilon] && \text{(noise variance)} \\ &+ (f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}}} [f_{S_{\text{train}}}(\mathbf{x}_0)])^2 && \text{(Squared bias)} \\ &+ \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(\mathbb{E}_{S_{\text{train}}} [f_{S_{\text{train}}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right], && \text{(Variance)}\end{aligned}$$

Bias-Variance Decomposition

$$\begin{aligned}
 \mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] &= \text{Var}_{\epsilon \sim \mathcal{D}_\epsilon} [\epsilon] && \text{(noise variance)} \\
 &+ (f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}}} [f_{S_{\text{train}}}(\mathbf{x}_0)])^2 && \text{(Squared bias)} \\
 &+ \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(\mathbb{E}_{S_{\text{train}}} [f_{S_{\text{train}}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right], && \text{(Variance)}
 \end{aligned}$$

This equality is the expected squared prediction error at the fixed input $\mathbf{x}_0$ under the stated assumptions. The noise variance is irreducible.

Bias-Variance Decomposition

$$\begin{aligned} \mathbb{E}_{S_{\text{train}} \in \mathcal{D}} [L(f_{S_{\text{train}}})] &= \text{Var}_{\epsilon \sim \mathcal{D}_{\epsilon}} [\epsilon] && \text{(noise variance)} \\ &+ (f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}}} [f_{S_{\text{train}}}(\mathbf{x}_0)])^2 && \text{(Squared bias)} \\ &+ \mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(\mathbb{E}_{S_{\text{train}}} [f_{S_{\text{train}}}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right], && \text{(Variance)} \end{aligned}$$

This equality is the expected squared prediction error at the fixed input $\mathbf{x}_0$ under the stated assumptions. The noise variance is irreducible.

⇒ In order to minimize the true error, we need to select a method that **simultaneously achieves low bias and low variance**.

Component 1: Noise $\text{Var}_{\epsilon \sim \mathcal{D}_\epsilon}[\epsilon]$ —a strict lower bound on what error we can achieve



Component 1: Noise $\text{Var}_{\epsilon \sim \mathcal{D}_\epsilon}[\epsilon]$ —a strict lower bound on what error we can achieve



- It is not possible to go below the noise level

Component 1: Noise $\text{Var}_{\epsilon \sim \mathcal{D}_\epsilon}[\epsilon]$ —a strict lower bound on what error we can achieve



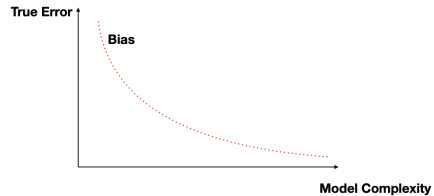
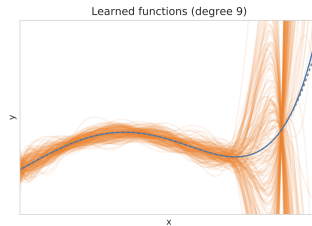
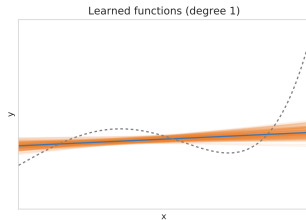
- It is not possible to go below the noise level
- Even if we know the true model f , we still suffer from $L(f) = \mathbb{E}[\epsilon^2]$

Component 1: Noise $\text{Var}_{\epsilon \sim \mathcal{D}_\epsilon}[\epsilon]$ —a strict lower bound on what error we can achieve

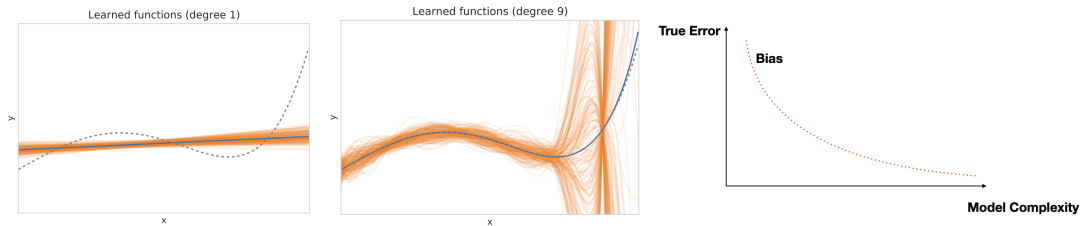


- It is not possible to go below the noise level
- Even if we know the true model f , we still suffer from $L(f) = \mathbb{E}[\epsilon^2]$
- It is not possible to predict the noise from the data since they are independent

Component 2: Bias $(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'} }(\mathbf{x}_0)])^2$

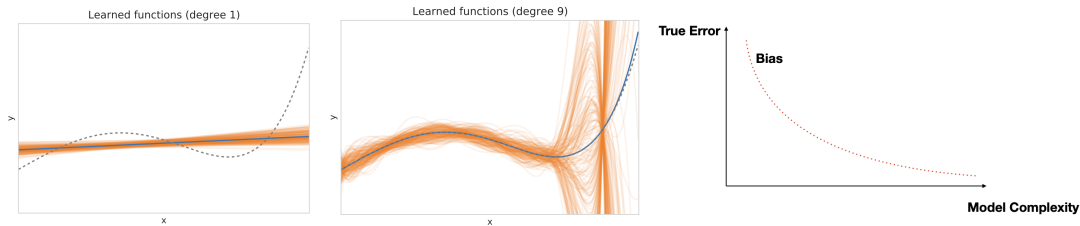


Component 2: Bias $(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'} }(\mathbf{x}_0)])^2$



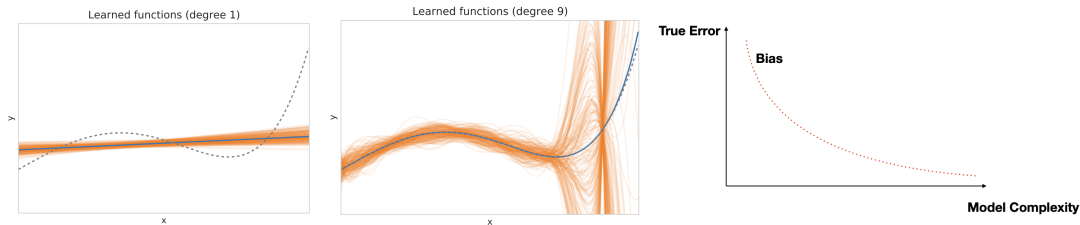
- It measures how far off in general the models' predictions are from the correct value.

Component 2: Bias $(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2$



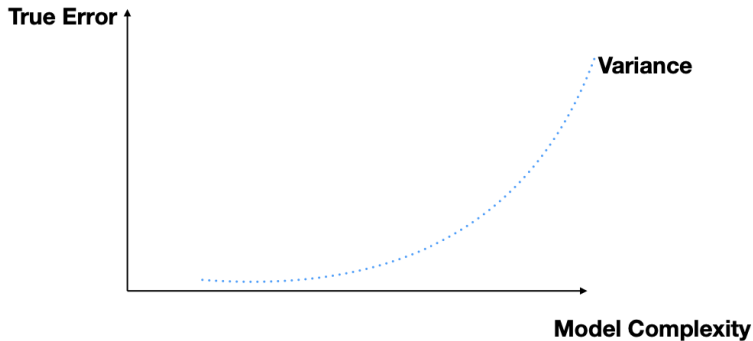
- It measures how far off in general the models' predictions are from the correct value.
- In this example, low complexity gives high squared bias.

Component 2: Bias $(f(\mathbf{x}_0) - \mathbb{E}_{S_{\text{train}'}} [f_{S_{\text{train}'}}(\mathbf{x}_0)])^2$

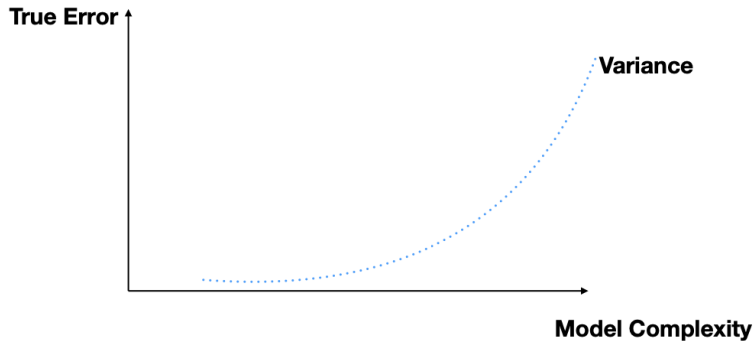


- It measures how far off in general the models' predictions are from the correct value.
- In this example, low complexity gives high squared bias.
- Increasing flexibility can reduce bias, although this is not automatic for every algorithm.

Component 3: Variance $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(\mathbb{E}_{S_{\text{train}}'} [f_{S_{\text{train}}'}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right]$

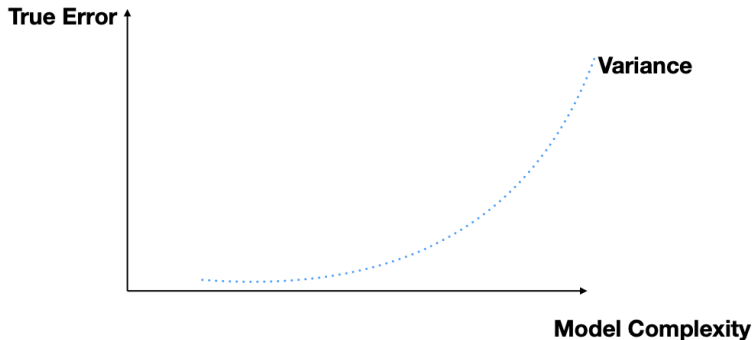


Component 3: Variance $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(\mathbb{E}_{S_{\text{train}}'} [f_{S_{\text{train}}'}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right]$



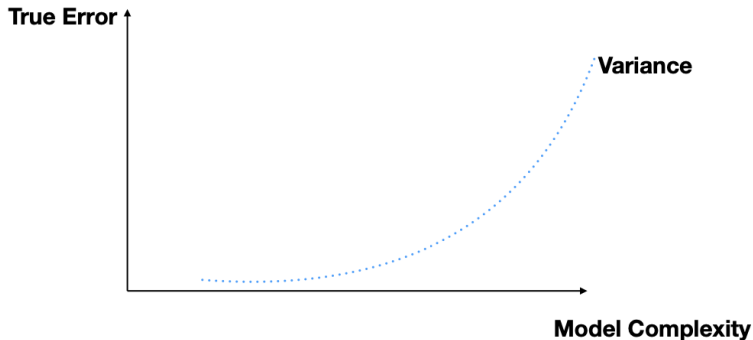
- Variance of the prediction function.

Component 3: Variance $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(\mathbb{E}_{S_{\text{train}}'} [f_{S_{\text{train}}'}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right]$



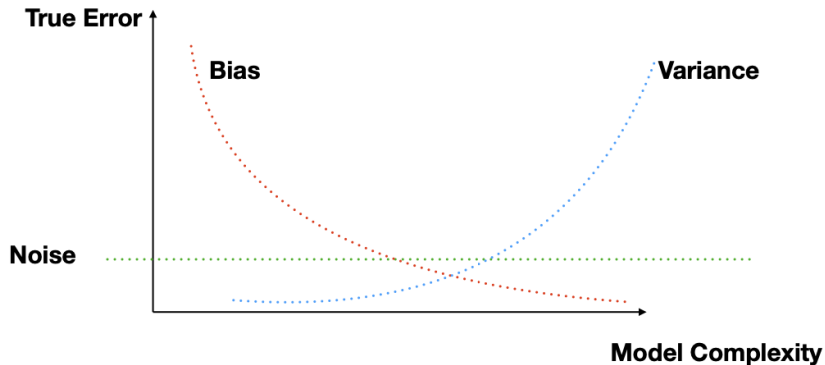
- Variance of the prediction function.
- It is **how much the predictions for a given point vary** between different realizations of the training set.

Component 3: Variance $\mathbb{E}_{S_{\text{train}} \sim \mathcal{D}} \left[(\mathbb{E}_{S_{\text{train}}'} [f_{S_{\text{train}}'}(\mathbf{x}_0)] - f_{S_{\text{train}}}(\mathbf{x}_0))^2 \right]$

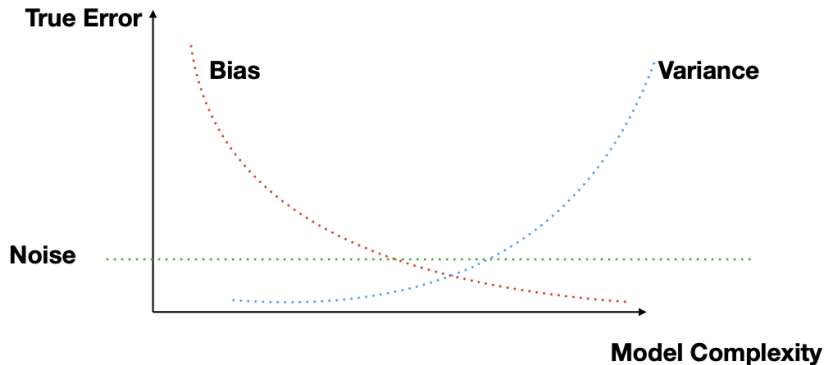


- Variance of the prediction function.
- It is **how much the predictions for a given point vary** between different realizations of the training set.
- If we consider complicated models, then small variations in the training set can result in large changes in the prediction.

Bias Variance tradeoff and U-shape curve

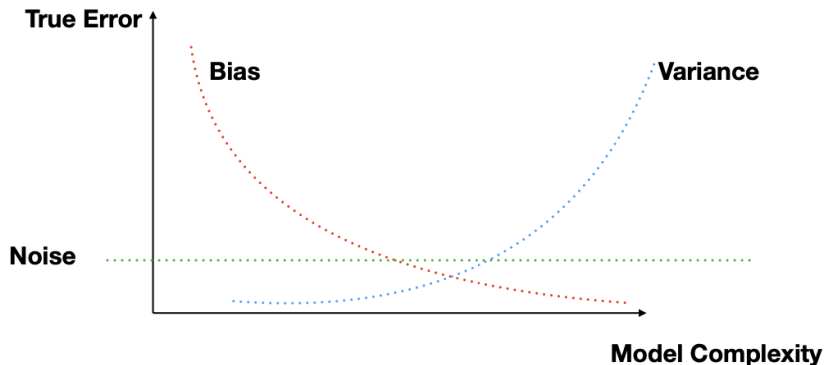


Bias Variance tradeoff and U-shape curve

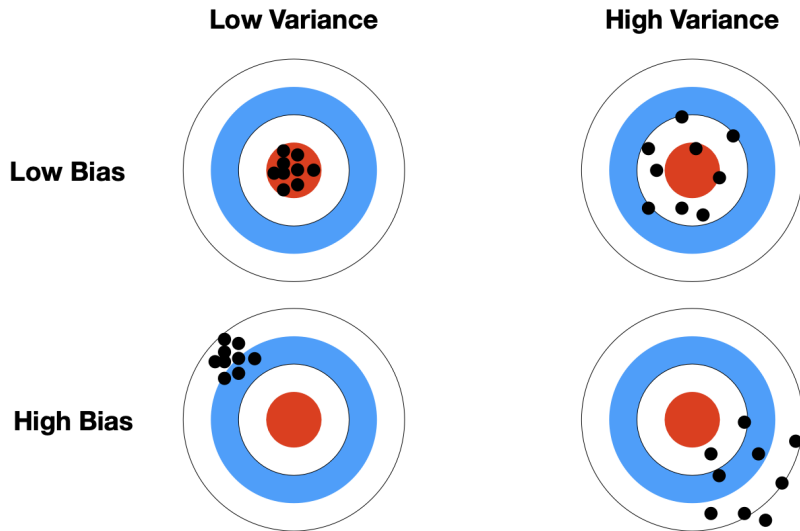


- If the complexity is too low, you cannot approximate well (under-fitting)

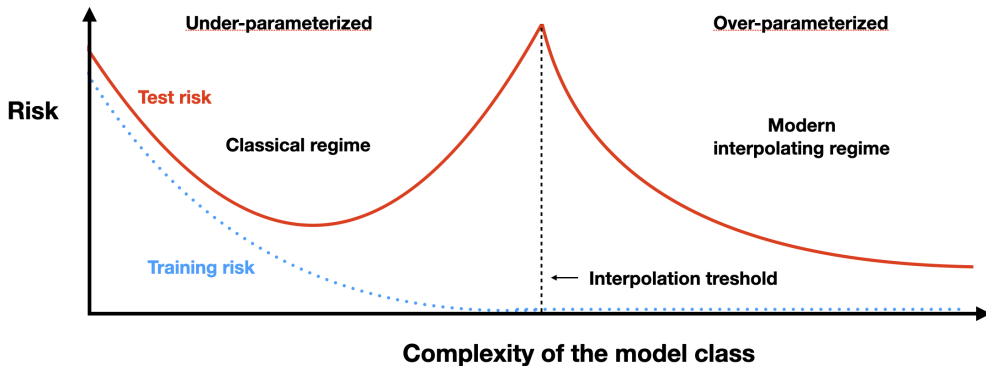
Bias Variance tradeoff and U-shape curve



- If the complexity is too low, you cannot approximate well (under-fitting)
- If the complexity is too large, you have a problem with the variance (over-fitting)



Optional: Double descent curve in Deep Learning



Three decisions to carry forward

1. Control flexibility:

- Ridge and lasso trade training fit for a preference toward smaller coefficients.
- The regularization strength λ is a hyperparameter, not a quantity learned by minimizing the same training loss.

Three decisions to carry forward

1. Control flexibility:

- Ridge and lasso trade training fit for a preference toward smaller coefficients.
- The regularization strength λ is a hyperparameter, not a quantity learned by minimizing the same training loss.

2. Separate the roles of data:

- Fit on training data, select on validation data, and evaluate once on test data.

Three decisions to carry forward

1. Control flexibility:

- Ridge and lasso trade training fit for a preference toward smaller coefficients.
- The regularization strength λ is a hyperparameter, not a quantity learned by minimizing the same training loss.

2. Separate the roles of data:

- Fit on training data, select on validation data, and evaluate once on test data.

3. Diagnose the failure mode:

- High bias suggests insufficient flexibility; high variance suggests sensitivity to the sample.

Training error tells us how well we fitted the observed sample. It cannot, by itself, answer any of these three decisions.